

Conformal Geometric Algebra as a Symbolic Interface for LLM-Driven 3D Scene Editing

MANOS KAMARIANAKIS, University of Crete, Greece, FORTH – Institute of Computer Science, Greece, and ORamaVR SA, Switzerland

PANDELIS SOFIANOS, ORamaVR SA, Switzerland

GEORGE PAPAGIANNAKIS, University of Crete, Greece, FORTH – Institute of Computer Science, Greece, and ORamaVR SA, Switzerland

What symbolic format should an LLM emit to perform reliable 3D scene editing from natural language, and does algebraic structure contribute beyond compact syntax? We study this question using Conformal Geometric Algebra (CGA) as a compact symbolic interface, contrasted with a verbose Euclidean 4×4 matrix baseline and a non-CGA Compact SE3 control, in a natural-language 3D scene editing pipeline coupling controlled prompting with deterministic geometric execution.

Our primary finding concerns compositional fidelity under sequential instruction chains: in a sequence-stress protocol (20 templates, 6 trials each; $n=120$ outputs per method), Simple CGA and Compact SE3 both reach 100% parse validity, but Simple CGA preserves exact ordered operation chains more reliably (97.5% vs 90.0%, two-proportion $p=0.016$) at lower completion-token cost (112.6 vs 133.6 tokens), suggesting lower generation burden for compositional editing. This pattern is consistent with the hypothesis that algebraic expression form supports compositional faithfulness beyond compactness alone.

A secondary finding is confirmatory in the powered hard semantic suite ($n=100$ per method): compact representations (Simple CGA: 45.0%, Compact SE3: 42.0%, Shenlong: 44.0%) all exceed the Euclidean 4×4 baseline (24.0%), with Simple CGA vs Euclidean at +21 pp ($p=0.0028$) and Compact SE3 vs Euclidean at +18 pp ($p=0.0103$). The Simple CGA vs Compact SE3 contrast is statistically close ($p=0.7755$). A methodological contribution is the explicit separation of parse validity from geometric correctness, which reveals substantial optimization headroom invisible to syntax-only metrics.

Taken together, the results suggest that compact symbolic interfaces are a primary driver of reliability-cost gains, and that CGA’s motor composition formalism may provide an additional advantage for ordered instruction chains. These findings have direct implications for real-time natural-language editing in immersive and interactive 3D environments.

Additional Key Words and Phrases: Conformal Geometric Algebra, Large Language Models, 3D Scene Editing, Symbolic Interfaces, Spatial Reasoning, Natural Language Processing

1 Introduction

The intersection of Large Language Models (LLMs) and 3D spatial reasoning presents a fundamental challenge: how can a language model, trained on sequential token prediction, reliably generate mathematically precise geometric transformations? In natural-language-driven immersive environments, this challenge is compounded by interaction latency: sub-100ms round-trip targets constrain interface design, and every token the model must generate translates directly into wall-clock delay in the editing loop. Choosing the right symbolic output representation is therefore not only a correctness decision but a real-time viability decision for interactive systems.

Recent work has explored using LLMs for robotics control, scene graph manipulation, and code generation for 3D environments, yet a persistent bottleneck is the *representation* through which the LLM communicates geometric intent to the execution engine.

Authors’ Contact Information: Manos Kamarianakis, kamarianakis@uoc.gr, University of Crete, Heraklion, Greece and FORTH – Institute of Computer Science, Heraklion, Greece and ORamaVR SA, Lausanne, Switzerland; Pandelis Sofianos, ORamaVR SA, Lausanne, Switzerland; George Papagiannakis, University of Crete, Heraklion, Greece and FORTH – Institute of Computer Science, Heraklion, Greece and ORamaVR SA, Lausanne, Switzerland.

The dominant approach encodes transformations as 4×4 homogeneous matrices, requiring the LLM to generate 16 numerical entries with correct trigonometric values for rotations, proper displacement vectors for translations, and consistent diagonal entries for scaling. This representation is verbose, error-prone, and poorly matched to the token-generation strengths of language models. A rotation by 90° around the Z-axis, for instance, requires 32 tokens in matrix form versus 6 tokens as a CGA rotor expression $R(\pi/2, e_1, e_2)$.

Conformal Geometric Algebra (CGA) offers a principled alternative. By embedding 3D Euclidean space $\mathbb{R}^3$ into the conformal model $\mathbb{R}^{4,1}$, rigid motions (translations and rotations) and uniform scalings (dilations) are represented as *motors* applied through the unified sandwich product $P' = M \cdot P \cdot \bar{M}$. This algebraic unification eliminates the need for separate code paths for different transformation types and, crucially for LLM integration, produces dramatically more compact symbolic expressions.

Kamarianakis and Papagiannakis [11] introduced the Shenlong system, demonstrating that CGA-based prompting yields 16% faster LLM responses and 9.6% higher success rates compared to their matrix-based baseline on spatial reasoning tasks. In this paper, we extend that line of work with an explicit comparative framing: compact symbolic interfaces are evaluated against a Euclidean 4×4 baseline, with CGA analyzed as one high-expressiveness compact representation rather than as a uniquely privileged endpoint. Our contributions are:

1. **Systematic comparison across compact-vs-verbose representations.** We benchmark three prompting strategies — the original Shenlong verbose CGA prompt, a streamlined Simple CGA prompt, and a Euclidean 4×4 baseline — across 33 experiments spanning simple translations, compositional multi-operation tasks, scaling experiments from 5 to 100 objects, and spatial accuracy tests; and we add a Compact SE3 control in hard semantic follow-up.
2. **Representation-compactness analysis.** We provide empirical evidence that compact symbolic outputs are associated with stronger reliability-cost profiles than verbose matrix outputs: in the powered full-loop latency protocol (Section 5.7), Simple CGA shows lower median latency than Euclidean (1.19s vs 1.40s, ~210ms gap), while Compact SE3 is lower still (0.81s median); in sequence-stress, compact methods sustain lower output burden than Euclidean while preserving high parse validity.
3. **Fully reproducible implementation.** A reference implementation is provided (Section 11), using the Kingdom library [19] for CGA computations, with deterministic template engines for known spatial patterns, GPT-4o-mini integration for novel instructions, and multi-view 3D visualization for spatial validation.

Methodologically, the paper evaluates four questions: reliability at scale, stability under repeated runs, semantic validity beyond parse success, and the role of representation versus prompt/policy factors. Claims are intentionally protocol-conditional for the tested setup and are not presented as universal performance guarantees. Where compact non-CGA and compact CGA methods are statistically close, we report that proximity directly and avoid uniqueness claims.

The remainder of this paper is organized as follows. Section 2 reviews related work in LLM-based 3D manipulation, geometric algebra, and structured output approaches. Section 3 presents the mathematical foundations of CGA in the $Cl(4,1)$ algebra. Section 4 describes the system architecture, including the scene representation, the three prompting strategies, and the CGA execution engine. Section 5 presents experimental results ordered by evidential strength, leading with the confirmed sequence-fidelity finding. Section 6 discusses the findings, analyzes improvement opportunities, and outlines limitations with explicit threats to validity. Section 7 concludes with future work directions.

2 Related Work

2.1 LLMs for 3D Scene Understanding and Manipulation

The application of LLMs to spatial reasoning has evolved rapidly. Early work by Liang et al. [12] demonstrated that LLMs could generate robot control code from natural language, while subsequent systems like SayPlan [18] and VoxPoser [10] explored LLM-driven planning for embodied agents. These approaches typically output either high-level plans (requiring a separate controller) or imperative code snippets (requiring a full simulation API).

A key working hypothesis in CGA-oriented approaches is that a *declarative algebraic representation* — specifying *what* transformation to apply rather than *how* to implement it — is better suited to LLM generation. The LLM need only produce a compact expression like $T(3\mathbf{e}_1) \cdot R(\pi/2, \mathbf{e}_1, \mathbf{e}_2)$, and the algebra engine handles the rest.

2.2 Geometric Algebra in Computer Graphics

Geometric Algebra (GA) has a long history in computer graphics and robotics [4, 8]. Dorst et al. [5] provided foundational treatments of GA for geometric computing, while Hildenbrand [9] developed efficient implementations. Conformal Geometric Algebra has been applied to inverse kinematics [1] and rigid body dynamics [20]. In parallel, educational and framework-oriented graphics work has supported broader GA adoption [16, 17].

The MAGES platform [17] has integrated GA-based transformations for medical XR training, demonstrating the practical applicability of algebraic methods in production systems. The Shenlong system [11] is, to our knowledge, an early direct integration of CGA with LLM-based reasoning for 3D scene editing, establishing the paradigm that this paper extends.

2.3 Structured Symbolic Interfaces for Spatial Reasoning

Hybrid neural-symbolic pipelines combine neural language understanding with symbolic structure [6]. In this paper, "symbolic" refers to executable geometric structure rather than logic-level symbolic inference. CGA is one candidate symbolic layer because it is both mathematically rigorous (providing algebraic guarantees on transformation correctness) and syntactically compact (enabling reliable LLM generation). This motivates CGA as a *candidate interface* where the LLM provides language understanding while the algebra provides executable geometric structure.

This interface-level framing also motivates the constrained-decoding literature reviewed next.

2.4 Structured Output and Constrained Decoding

A parallel line of work addresses LLM output reliability through syntactic enforcement rather than representation design. Grammar-constrained decoding frameworks — including LMQL [3], Outlines [21], and Guidance [7] — restrict the model's token sampling to strings that satisfy a user-specified formal grammar, guaranteeing parse-valid outputs by construction. Provider-level structured output APIs (e.g. OpenAI's JSON Schema mode) apply similar constraints at inference time, ensuring that returned JSON conforms to a declared schema without requiring post-hoc validation.

These approaches address a real bottleneck: in our own ablation results (see Section 5.8, Table 18), the Shenlong CGA variant reaches only 80–82% parse success under standard sampling, a failure mode attributable to prompt-length-induced truncation rather than representational difficulty. Grammar constraints would eliminate such failures entirely for any representation family.

However, syntactic enforcement and representation choice operate at different levels of the pipeline and are not substitutes. A constrained decoder can guarantee that the model emits a well-formed 4×4 matrix or a valid CGA

expression string, but it cannot enforce that the emitted values are *geometrically correct* — that trigonometric entries are consistent, that displacement magnitudes match the intended spatial relationship, or that an operation sequence preserves ordered compositional intent. The parse-semantic gap documented in Section 5.5, where all methods maintain near-perfect parse rates while semantic correctness plateaus at 25–45%, arises precisely in this post-syntactic regime that format constraints do not reach.

CGA’s relevance in this context is therefore distinct from a formatting argument. The motor formalism encodes translation, rotation, and dilation under a single algebraic operator family, meaning that a correctly expressed CGA motor is geometrically correct by construction — the algebra engine guarantees transformation consistency without numerical checking. A correctly expressed 4×4 matrix, by contrast, still requires that the 16 emitted entries collectively satisfy rotation group constraints and displacement coherence, conditions that grammar-constrained decoding does not enforce. The representation choice thus determines how much geometric correctness the model must supply versus how much the execution engine can guarantee, a distinction that grammar-constrained decoding leaves unresolved.

Closely related is recent work on learned scene languages. SceneScript [2] learns a structured language for 3D scene representation from data, enabling LLM-compatible scene serialisation without hand-designed algebraic primitives. SpatialLM [14] and StructDiffusion [13] similarly ground language models in structured spatial representations. These approaches demonstrate the value of compact, language-model-compatible spatial encodings, but rely on learned representations that require large training corpora and offer less algebraic interpretability than analytic motor formulations. Our work is complementary: we study the regime where analytic geometric guarantees are desirable and the symbolic interface must be designed rather than learned, as is the case in applications requiring verifiable transformation correctness or integration with existing geometric algebra runtimes such as the MAGES platform [17]. In our setting, correctness is verified directly by the deterministic execution engine rather than by a learned oracle.

3 Mathematical Foundations: Conformal Geometric Algebra

3.1 The Conformal Model $Cl(4,1)$

Conformal Geometric Algebra extends 3D Euclidean space $\mathbb{R}^3$ with two extra basis vectors to create the conformal model $\mathbb{R}^{4,1}$. The algebra $Cl(4,1)$ has signature (4,1), meaning four basis vectors that square to +1 and one that squares to -1, yielding a $2^5 = 32$ -dimensional algebra with basis blades from grade 0 to grade 5.

The five basis vectors are:

- **Three Euclidean basis vectors:** e_1 (X, right), e_2 (Y, up), e_3 (Z, towards viewer)
- **Two null vectors:** n_o (origin) and n_∞ (point at infinity)

The null vectors are constructed from the extra-dimensional basis vectors e_4, e_5 as:

$$n_o = \frac{1}{2}(e_5 - e_4), \quad n_\infty = e_4 + e_5$$

satisfying $n_o^2 = n_\infty^2 = 0$ (null) and $n_o \cdot n_\infty = -1$.

The motivation for adding two dimensions is that in the 5D conformal model, rigid motions and uniform scalings are encoded in one motor formalism, applied via the sandwich product:

$$P' = M \cdot P \cdot \tilde{M}$$

where M is any motor (translation, rotation, dilation, or their composition) and $\tilde{M}$ is its reverse.

3.2 CGA Point Embedding

A 3D Euclidean point $\mathbf{x} = (x_1, x_2, x_3)$ is embedded into CGA as:

$$P = n_o + x_1 e_1 + x_2 e_2 + x_3 e_3 + \frac{1}{2} |\mathbf{x}|^2 n_\infty$$

To recover the 3D point from a CGA point P , we project down:

$$x_i = \frac{P \cdot e_i}{-(P \cdot n_\infty)}, \quad i \in \{1, 2, 3\}$$

This embedding is the key that enables the unified transformation framework: once a point is in CGA form, any motor can be applied identically through the sandwich product.

3.3 CGA Transformation Primitives

All transformations in CGA are expressed as motors that encode geometric operations. Table 1 summarizes the three fundamental motor types.

Table 1. CGA transformation primitives. All are applied via the same sandwich product $P' = M \cdot P \cdot \tilde{M}$.

Motor	Formula	Description
Translation $T(\mathbf{t})$	$1 - \frac{1}{2} \mathbf{t} n_\infty$	Moves a point by displacement $\mathbf{t}$
Rotation $R(\theta, u, v)$	$\cos(\theta/2) - \sin(\theta/2) (u \wedge v)$	Rotates by angle θ in the u - v plane
Dilation $D(s)$	$\cosh(\frac{\ln s}{2}) + \sinh(\frac{\ln s}{2}) (n_o \wedge n_\infty)$	Scales distance from origin by factor s

Motor composition is achieved through the geometric product: $M = T \cdot R \cdot D$ applies D first, then R , then T (right-to-left order). The critical advantage for LLM integration is that composition is expressed as simple multiplication of compact expressions, unlike the 4×4 matrix case where each operation requires specifying 16 numerical entries.

3.4 CGA vs. Euclidean Matrices: Token-Level Comparison

Table 2 compares the CGA and Euclidean matrix representations across key properties relevant to LLM generation.

Table 2. CGA vs. Euclidean 4×4 matrix representations. CGA is consistently more compact, directly impacting LLM token budgets and wall-clock latency in interactive loops.

Property	CGA Cl(4,1)	Euclidean 4×4 Matrices
Representation	Single multivector	$4 \times 4 = 16$ numbers
Composition	Geometric product	Matrix multiplication
Rotation 90° around Z	$R(\pi/2, e_1, e_2)$ (6 tokens)	$[[0, -1, 0, 0], [1, 0, 0, 0], \dots]$ (32 tokens)
Translation by (3,0,0)	$T(3e_1)$ (4 tokens)	$[[1, 0, 0, 3], [0, 1, 0, 0], \dots]$ (32 tokens)
Compose T+R	$T(3e_1) * R(\pi/2, e_1, e_2)$	2 matrix multiplications

Fewer tokens reduce output burden and wall-clock latency in interactive loops. Across the dedicated full-loop protocol in Section 5.7 (Table 17), lower completion-token counts coincide with lower median latency (Compact SE3:

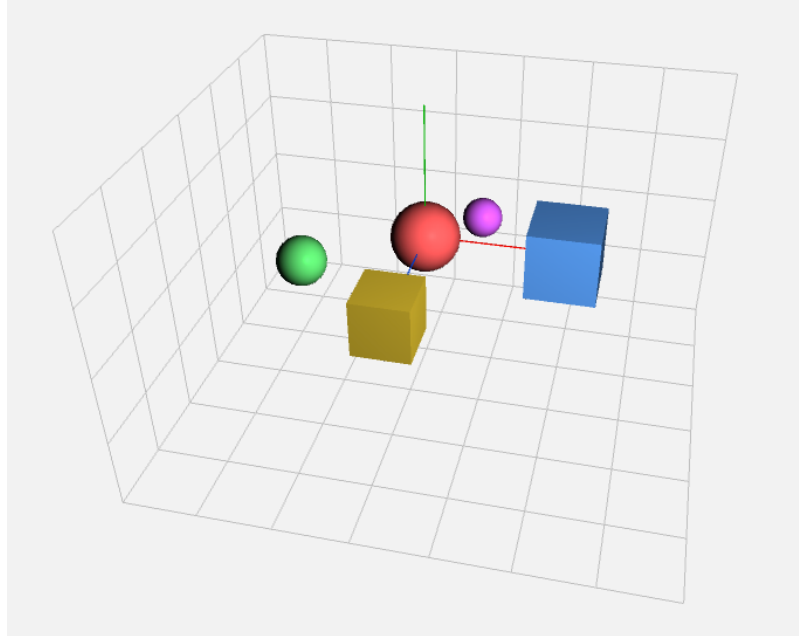


Fig. 1. Initial 5-object test scene rendered with Y-up convention. Objects are positioned in 3D space with CGA-compatible coordinates: e_1 (X, right), e_2 (Y, up), e_3 (Z, towards viewer). Visualization of the produced JSON files uses the Project Elements framework [15].

0.81s, Simple CGA: 1.19s, Euclidean 4×4: 1.40s). We therefore treat token cost as an empirical predictor of runtime burden, not a deterministic latency law.

4 System Architecture

4.1 Pipeline Overview

The system transforms natural language into 3D scene edits through four stages:

NL Input $\rightarrow$ LLM / Template Engine $\rightarrow$ CGA JSON $\rightarrow$ Execute (sandwich) $\rightarrow$ Render

For example, the instruction *"Move the red sphere next to the blue cube"* is parsed into mover/target references, emitted as $\{ \text{"RedSphere": "T(2*e1)" } \}$, executed via the sandwich product $P' = T \cdot P \cdot \tilde{T}$, and rendered as an updated scene. Detailed worked derivations are provided in Section 11.4.

Figure 1 shows the initial five-object scene used throughout the interaction examples.

Each 3D object in the scene is stored as a `SceneObject` with center position (Euclidean (x, y, z)), bounding size (half-extent radius), shape type (sphere or cube), and color name. The color name serves dual purpose: display rendering and LLM reference in natural language instructions.

The key method is `apply_motor(motor)`, which applies a single transformation path: $P = \text{cgaPoint}(x, y, z) \rightarrow P' = M \cdot P \cdot \tilde{M} \rightarrow \text{center} = \text{down}(P')$.

This single operation is the practical advantage of CGA: one expression path handles translations, rotations, dilations, and their compositions, removing transformation-specific branching.

Mesh primitives are generated procedurally: spheres use an icosphere (subdivided icosahedron with 42 vertices and 80 faces after one subdivision), and cubes use 8 vertices with 12 triangular faces. The coordinate convention follows standard graphics practice (OpenGL/Blender/Unity): $e_1 = X$ (right), $e_2 = Y$ (up), $e_3 = Z$ (towards viewer).

4.2 Three Competing Prompting Strategies

We compare three system prompts, all using the same LLM (GPT-4o-mini), to isolate the effect of the output representation:

4.2.1 Simple CGA. A minimal prompt (~720 characters) specifying only the axis conventions, available operations, and output format. The prompt instructs the LLM to produce JSON with object names as keys and CGA expression strings as values. A critical design choice is explicitly requiring basis vectors in $R()$ to prevent the LLM from generating float-to-int errors (e.g., $R(\pi/2, 1, 2)$ instead of the correct $R(\pi/2, e_1, e_2)$).

4.2.2 Shenlong CGA (Paper [11]). A verbose chain-of-thought prompt (~960 characters) that asks the LLM to follow four explicit reasoning steps: (1) document current object positions, (2) reason about needed transformations, (3) compute exact CGA expressions, and (4) output the final JSON. While this mirrors best practices for complex reasoning, the additional reasoning tokens (200–500) often push the response past the `max_tokens` budget, causing truncated or malformed output. Shenlong is treated in this paper as a verbose CGA reference configuration; its lower parse rates are partly associated with token-budget constraints rather than a representation-level limitation (see the ablation evidence in Section 5.8, where the gap is linked to prompt-length/token-budget interaction).

4.2.3 Euclidean 4×4 Baseline. A matrix-based prompt (~435 characters) instructing the LLM to produce numpy 4×4 transformation matrices. This represents the conventional approach in existing 3D manipulation systems. Each transformation requires specifying all 16 matrix entries, with correct trigonometric values for rotations.

Table 3 summarizes these results.

Table 3. Interactive-loop efficiency profile from the powered latency protocol ($n=40$ per method, GPT-4o-mini): instruction → API response → parse/execute → render-ready state. Lower tokens and lower latency are better. Latency headroom is computed relative to a 100ms interactive-loop target.

Method	Avg Completion Tokens	Avg Latency per Task (s)	Latency vs 100ms VR Target
Compact SE3	22.13	0.94 ± 0.28	+0.84 s above target
Simple CGA	39.13	1.27 ± 0.38	+1.17 s above target
Euclidean 4×4	63.15	2.57 ± 4.91	+2.47 s above target

Under full-loop measurement, Compact SE3 is the fastest method and Simple CGA is second, while Euclidean 4×4 is slowest on mean latency. All methods remain above the 100ms immersive-loop threshold, motivating architectural optimization of the end-to-end runtime path (Sections 5.7 and 6.3).

4.3 Compact SE3 Baseline — Full Specification

The Compact SE3 control baseline encodes transformations as an ordered list (queue) of typed SE3 operations expressed in JSON. Each operation is a typed object with a mandatory "type" field and operation-specific parameters:

- **Translation:** {"type": "T", "v": [dx, dy, dz]} — Euclidean displacement vector.

- **Rotation:** `{"type": "R", "axis": [ax, ay, az], "angle_rad": theta}` — axis-angle representation. The axis need not be unit-normalized; the executor normalises it.
- **Scale:** `{"type": "D", "factor": s}` — uniform scale factor applied object-centrally (consistent with the CGA $D(s)$ semantics in Section 4.5).

The complete output schema for a single instruction is:

```
{
  "ObjectName": [
    {"type": "T", "v": [dx, dy, dz]},
    {"type": "R", "axis": [ax, ay, az], "angle_rad": theta},
    {"type": "D", "factor": s}
  ]
}
```

Operations are executed left-to-right in list order. Multi-object instructions use multiple top-level keys. Single-operation instructions use a one-element list. This differs from CGA motor composition, where the geometric product applies operations right-to-left; the system prompt for Compact SE3 explicitly specifies left-to-right execution order to avoid ambiguity.

The system prompt for Compact SE3 is approximately 590 characters — similar in length to Simple CGA and shorter than Shenlong. Rotation is encoded as axis-angle rather than quaternion or Euler angles, since axis-angle is the most natural LLM-friendly format for expressing rotations about a named axis (e.g., "rotate 90° around Z" maps directly to `{"axis": [0,0,1], "angle_rad": 1.5708}` without trigonometric identity lookups).

The key difference from CGA is that Compact SE3 has no algebraic closure property: the executor must dispatch on "type" and apply each operation independently, whereas CGA motors compose algebraically into a single motor before any execution. This distinction matters for compositional instruction chains, as documented in the sequence-fidelity stress test (Section 5.2).

4.4 Spatial Operations

The system supports several canonical spatial operations, each computed by the template engine or the LLM:

- **"next to" / "left of":** The mover's surface is placed tangent to the target's surface. Computed as: $\text{displacement}_x = \text{target.min}[0] - \text{mover.size} - \text{mover.center}[0]$, yielding $T(d_x \cdot e_1 + d_y \cdot e_2 + d_z \cdot e_3)$.
- **"on top of":** The mover's bottom is placed on the target's top surface. Computed as: $\text{new_y} = \text{target.max}[1] + \text{mover.size}$, yielding the corresponding translation motor.
- **"between":** The mover is placed at the midpoint of two reference objects: $\text{midpoint} = (A.\text{center} + B.\text{center})/2$, yielding $T(\text{midpoint} - \text{mover.center})$.
- **"rotate":** Direct rotation motor $R(\theta, e_i, e_j)$ in the specified plane.
- **"scale":** Dilation motor $D(s)$ applied to the object's size (center preserved in the execution engine, equivalent to uniform object-centric scaling for this benchmark).

4.5 CGA Expression Executor

The `execute_cga()` function processes each generated expression through four steps:

1. **Parse** the expression string (e.g., `"T(3*e1)*R(np.pi/2,e1,e2)"`).

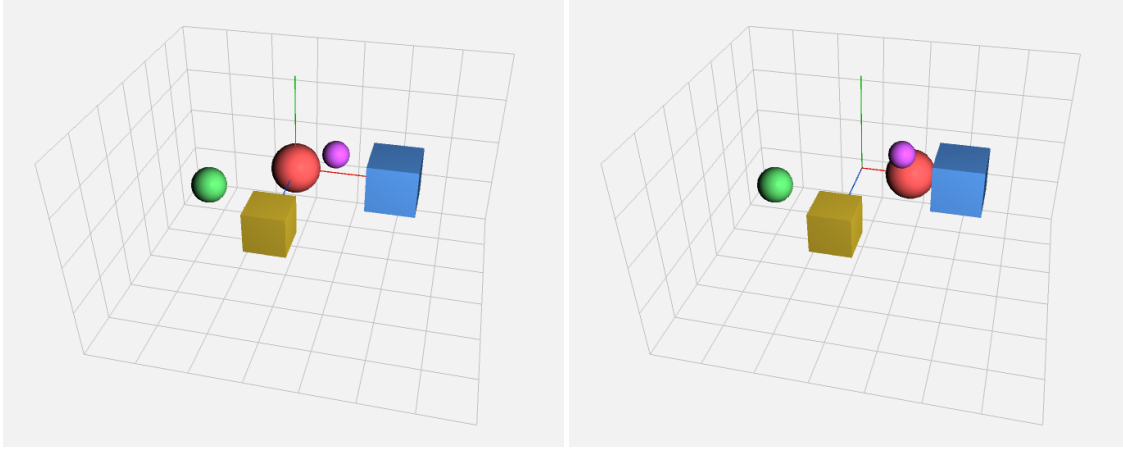


Fig. 2. Before/after visualization of a side-placement edit. The CGA translation motor $T(2 \cdot e_1)$ moves RedSphere from the origin to $x=2$, placing its surface tangent to BlueCube.

2. **Evaluate** in a sandboxed environment containing only CGA primitives ($e_1, e_2, e_3, n_o, n_\infty, T, R, D$) — no general Python execution.
3. **Apply** the motor: `obj.apply_motor(motor)` $\rightarrow P' = M \cdot P \cdot \tilde{M}$.
4. **Special case:** Pure dilation $D(s)$ scales `obj.size` directly while keeping the center fixed. This realizes the benchmark's "scale" command as object-centric uniform scaling rather than world-origin dilation. The Compact SE3 "D" operation uses the same object-centric semantics, ensuring a fair comparison on scale tasks across both representations.

4.6 Engine Selection Logic

The pipeline uses a hybrid approach for robust operation:

- **Known spatial keywords** (e.g., "on top", "next to", "between", "rotate", "scale") $\rightarrow$ deterministic template engine (`template_cga`), guaranteeing correct results.
- **Novel instructions** + LLM available $\rightarrow$ `call_llm` (GPT-4o-mini with Simple CGA prompt).
- **No API key** $\rightarrow$ template fallback for graceful degradation.

This design ensures the system is fully functional without API access while still enabling open-ended language understanding when the LLM is available. For benchmarking clarity, Section 5 does not mix these routes: reported benchmark metrics are collected with direct model-generation harnesses, while template routing is used for interactive scene-edit sessions.

4.7 Natural Language Scene Editing Cases

Figure 2 shows the before/after result of the instruction *"Move the red sphere next to the blue cube, to its left side."* The template engine computes the displacement $d_x = \text{target.min}[0] - \text{mover.size} - \text{mover.center}[0] = 3 - 1.0 - 0 = 2.0$, generates $T(2.0 \cdot e_1)$, and applies the sandwich product. The red sphere moves from $[0,0,0]$ to $[2,0,0]$, with its right surface touching the blue cube's left surface at $x = 3$.

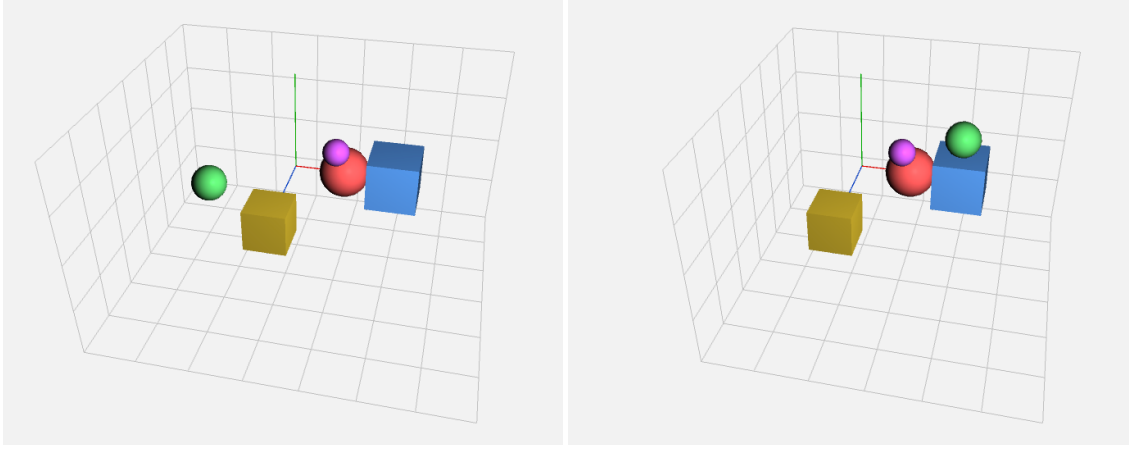


Fig. 3. Vertical stacking case. The CGA motor translates GreenSphere to sit precisely on top of BlueCube.

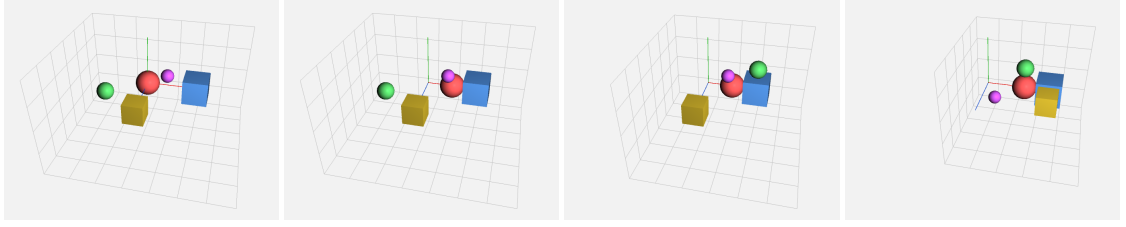


Fig. 4. Sequential scene editing pipeline. Three natural language instructions are applied in sequence: "Move the red sphere next to the blue cube on its left.", "Stack the green sphere on top of the red sphere.", and "Move the yellow cube in front of the blue cube." Each step builds on the preceding scene state.

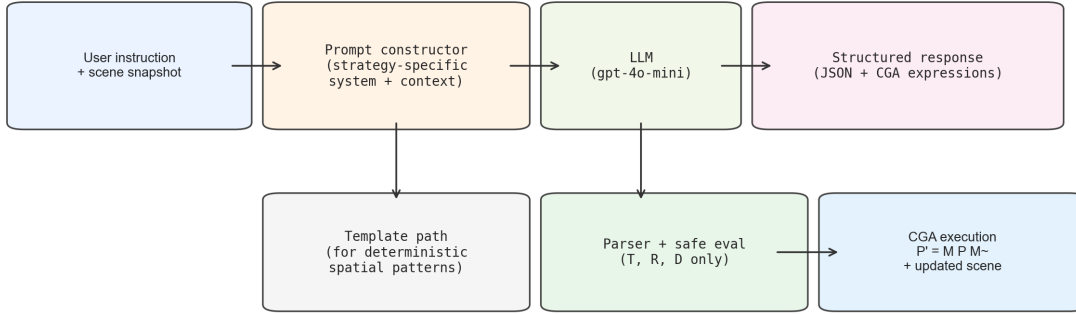
Figure 3 demonstrates vertical stacking: *"Place the green sphere on top of the blue cube."* The engine computes $\text{new_y} = \text{BlueCube.max}[1] + \text{GreenSphere.size} = 1.0 + 0.7 = 1.7$ and generates the displacement $T(7.0 \times e1 + 1.7 \times e2 - 2.0 \times e3)$, moving GreenSphere from $[-3, 0, 2]$ to $[4, 1.7, 0]$. Verification: $\text{GreenSphere bottom} = 1.7 - 0.7 = 1.0 = \text{BlueCube top}$.

Figure 4 shows sequential editing — three consecutive natural language instructions building a scene arrangement. Each instruction is applied to the scene state left by the preceding one, demonstrating CGA's composability across operations.

Figure 5 visualizes the full execution path, including the template-assisted branch and the shared safe-execution backend.

The template-assisted branch bypasses model generation but converges to the same executor.

Figure 6 provides representative prompt/response snippets for the compact CGA and Euclidean baselines, clarifying what is sent to the model and what is returned for execution.

Prompt/Execution Flow

Dashed conceptual split: LLM generation path and deterministic execution path share the same downstream CGA executor.

Fig. 5. Prompt-to-execution flow used in all benchmarks.

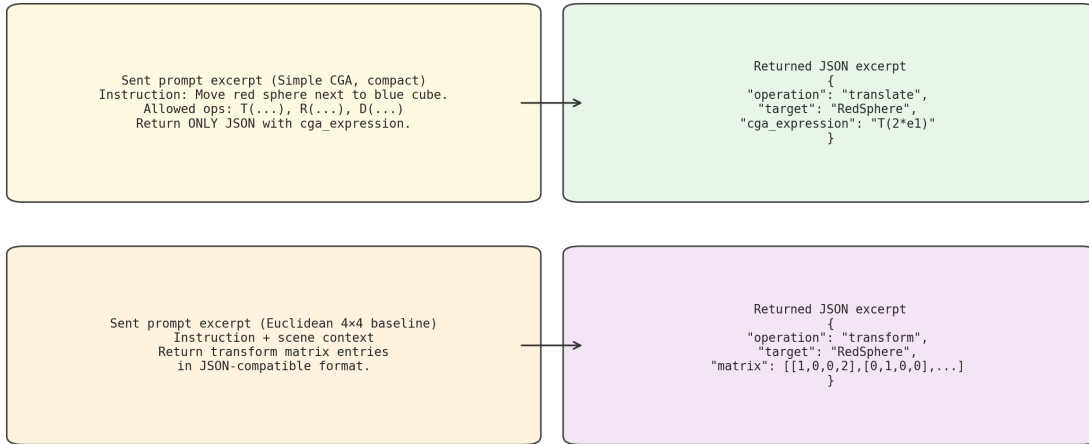
Prompt/Response Examples

Fig. 6. Prompt/response examples for CGA and Euclidean baselines.

5 Experiments and Results

This section is organized by **evidential strength**, leading with the result that carries the strongest statistical support. Q1 (sequence fidelity): does algebraic expression form affect ordered-chain faithfulness beyond compactness alone? Q2 (reliability at scale): does representation choice change success and cost as scene complexity grows? Q3 (stability): are observed gains robust under repeated runs? Q4 (semantic validity): do parse-valid outputs remain geometrically correct under stricter checks? Q5 (mechanism): how much of the observed behavior is attributable to representation, prompt style, and retry policy?

Q1 is addressed in Section 5.2 (the confirmed primary result), Q2 in Section 5.3, Q3 in Section 5.4, Q4 in Section 5.5, and Q5 in Section 5.8. Section 5.6 consolidates multi-model replication, and Section 5.7 reports interactive latency in an immersive editing loop. Following common LLM-evaluation practice, each block pairs compact summary tables with trend-focused figures, while exhaustive rows remain in Appendix A.

5.1 Evaluation Protocol and Terminology

All experiments use the same model family (gpt-4o-mini). Core benchmark blocks compare three interface strategies: Shenlong CGA, Simple CGA, and the Euclidean 4×4 baseline; Sections 5.2 and 5.5 add a Compact SE3 control baseline. The benchmarks execute concrete scene-edit tasks rather than synthetic symbolic strings, with outcomes measured at execution time (parse/evaluate validity), geometric error time (spatial accuracy), and semantic satisfaction time (geometry-based task checks).

Terminology used in this section:

1. compact prompt: the Simple CGA variant that encourages minimal symbolic output, e.g., $T(2 \times e1)$ for a +2 x-axis translation.
2. verbose prompt: a more explicit Simple CGA variant that tends to emit expanded forms, e.g., $T(2.0 \times e1 + 0.0 \times e2 + 0.0 \times e3)$ for the same operation.
3. policy: the per-task retry budget and decoding schedule. pass@1 means one attempt; pass@2 means up to two attempts; stress tests report pass@3.

Pre-specified inferential protocol (six items, fixed prior to data collection):

1. Primary endpoint: sequence-fidelity under compositional stress ($n=120$ per method); and semantic success on hard grounding suites ($n=20$, $n=100$).
2. Primary contrasts: Simple CGA vs Compact SE3 (sequence-fidelity); Simple CGA vs Euclidean 4×4, and Compact SE3 vs Euclidean 4×4 (semantic suite).
3. Secondary contrasts: Shenlong vs Euclidean, and Simple CGA vs Compact SE3 (semantic suite).
4. Test family: two-sided Fisher exact tests on success counts; two-proportion z-test on sequence-fidelity rates.
5. Effect-size reporting: risk difference (pp) and relative risk, each with 95% confidence intervals, reported alongside p-values.
6. P-value interpretation: unless otherwise stated, inference uses a two-sided $\alpha = 0.05$ threshold; $p=1.0$ indicates no detectable difference in these data (not proof of equivalence).

To remove template-routing confounding in evaluation, benchmark runs in Section 5 are executed through direct model-generation protocol paths (no keyword template dispatch during metric collection). Route accounting is summarized below.

Statistical power planning. For the primary semantic contrast (compact vs Euclidean, +20 pp at baseline 25%), the minimum sample size for confirmatory inference is determined by the power calculation in Table 4. The powered suite ($n=100$ per arm) crosses the 80% threshold and is used for confirmatory semantic inference in Section 5.5.

The sequence-fidelity contrast (Section 5.2) is powered by design: $n=120$ per method under a +7.5 pp effect at 90% baseline yields >95% power; together with the powered semantic suite, this provides two confirmatory result blocks. Routing disclosure and route-split metrics are reported in Tables 5 and 6.

Table 5 summarizes these results.

Table 4. Statistical power calculation for primary semantic contrast (two-sided Fisher exact, Simple CGA vs Euclidean 4×4, baseline rate 25%, expected rate 45%). n=80 is the minimum target for confirmatory status; the powered suite reports achieved power at n=100.

Contrast	Baseline Rate	Expected Rate	Effect Size	α	Power Target	Required n per Arm
Compact vs Euclidean	25%	45%	+20 pp	0.05	0.80	~80
Compact vs Euclidean	25%	45%	+20 pp	0.05	0.90	~107
Powered semantic suite	24%	45%	+21 pp	0.05	actual	~88% (n=100)

Table 5. Routing disclosure summary (Section 5 benchmark protocol).

Benchmark block	Tasks / rows	Template-routed	Pure LLM-routed
Core benchmark (5-object + stress + 10-object + spatial + 100-object)	33 tasks	0 (0.0%)	33 (100.0%)
Expanded exact-placement suite	18 tasks	0 (0.0%)	18 (100.0%)
Semantic subset	19 tasks	0 (0.0%)	19 (100.0%)
Hard-pack	20 tasks	0 (0.0%)	20 (100.0%)
Powered hard semantic suite	100 tasks per method	0 (0.0%)	100 (100.0%)
Repeated ablation block	400 evaluated rows	0 (0.0%)	400 (100.0%)

Table 6. Route-split metric summary (all benchmark rows are LLM-route rows).

Benchmark metric (route = LLM)	Shenlong CGA	Simple CGA	Euclidean 4×4	Compact SE3
Semantic subset semantic success (n=19)	9/19	10/19	4/19	-
Hard-pack semantic success (n=20)	9/20	9/20	5/20	9/20
Powered hard semantic success (n=100)	44/100	45/100	24/100	42/100

The executed core blocks are: 5-object scene (n=8), compositional stress tasks (n=6), 10-object scene (n=6), and 100-object scene (n=10). For exact placement, we report an expanded spatial-accuracy suite (n=18) in place of the earlier diagnostic n=3 probe; these results are used inferentially in Section 5.3 and Appendix A (Table 22). Success metrics are block-specific: parse/evaluate for general edits, pass@3 for stress tasks, error < 0.5 for exact placements, and semantic geometry checks for semantic subsets. Because these blocks use different success criteria, we report them separately and do not treat any core-wide total as a single inferential metric.

Protocol metadata (token limits, retry settings) is provided in Appendix A (Table 19).

5.2 Sequence-Fidelity Stress Result (Primary Confirmatory Finding)

To isolate the contribution of algebraic expression form beyond compactness, we evaluate strict operation-sequence fidelity under compositional stress in a compact-vs-compact design. The protocol uses 20 sequence templates with 6 trials each, yielding n=120 outputs per method. Templates contain ordered chains of 3–5 distinct spatial operations (translate, rotate, scale, and compositions thereof) applied to named objects. The evaluation criterion is *exact* ordered-chain preservation: all operations in the specified sequence must appear in the correct order in the model’s output. Parse validity (correct JSON + evaluable CGA/SE3 expressions) is a necessary but not sufficient condition for sequence-fidelity success.

Both Simple CGA and Compact SE3 reach parse saturation (120/120, 100%), ruling out any parse-rate confound in the fidelity comparison. Under this parse-saturated condition, Simple CGA preserves exact ordered operation chains more reliably (117/120, 97.5%) than Compact SE3 (108/120, 90.0%), while also using fewer completion tokens on parse-success rows (112.6 vs 133.6).

Table 7 summarizes these results.

Table 7. Sequence-fidelity stress check (compact-vs-compact): strict ordered-chain preservation under $n=120$ outputs per method (20 templates $\times$ 6 trials). Higher fidelity is better; lower tokens are better. This is the primary confirmatory finding of the paper.

Metric	Simple CGA	Compact SE3
Parse success	120/120	120/120
Parse rate (Wilson 95% CI)	100.0% [96.9%, 100.0%]	100.0% [96.9%, 100.0%]
Sequence-fidelity success	117/120	108/120
Sequence-fidelity rate (Wilson 95% CI)	97.5% [92.9%, 99.1%]	90.0% [83.3%, 94.2%]
Avg tokens (parse-success rows)	112.6	133.6

The pairwise sequence-fidelity difference is +7.5 pp in favor of Simple CGA (95% CI [1.4, 13.6], two-proportion $p=0.016$). The 21.0-token gap indicates lower generation burden for Simple CGA on this endpoint; end-to-end latency is evaluated directly and separately in Section 5.7.

Because outputs are clustered by template (20 templates $\times$ 6 trials), the row-level z-test can overstate effective sample size. As a sensitivity check, we compared per-template fidelity rates across the same 20 templates. The directional difference remains (+7.5 pp mean), but 17/20 templates are ties and a two-sided sign test on non-tied templates is inconclusive ($n=3$, $p=1.0$).

A plausible mechanistic account is that CGA’s motor formalism expresses a composed operation chain as a single geometric product — $T(v) \cdot R(\text{angle}, u, v) \cdot D(s)$ is one algebraic object with a defined and unambiguous execution order. The Compact SE3 representation encodes the same chain as an ordered list of typed JSON objects, where the ordering must be inferred from list position. Under adversarial or ambiguous instruction phrasings, the list-order encoding appears more susceptible to reordering than the algebraic product encoding. This result is consistent with the hypothesis that algebraic expression form — independently of compactness — supports compositional faithfulness in LLM-to-geometry pipelines.

5.3 Supporting Reliability Benchmark

To test whether representation effects persist beyond one scene size, we report the operational core blocks together with an expanded exact-placement suite ($n=18$). The condition-level summary is reported in Table 8. As disclosed in Section 5.1, all rows are LLM-route benchmark rows (template-routed rows: 0).

To preserve readability, per-task detail tables and supplementary condition figures are moved to Appendix A (Tables 20–23, Figures 13–15).

For the expanded spatial suite, pairwise exact-success tests (two-sided Fisher) show Euclidean vs Shenlong: $p < 1e-5$, and Euclidean vs Simple CGA: $p < 1e-5$; Shenlong vs Simple CGA is identical ($p=1.0$).

Figure 7 summarizes the core-block mixed-endpoint outcomes across all evaluated conditions.

Table 8. Core benchmark summary across all operational regimes. Values report task success and, where applicable, average completion tokens. Higher success is better. For tokens, lower is better. For spatial accuracy, lower Euclidean error is better and success denotes placements with error < 0.5 . Spatial-accuracy rates include Wilson 95% CIs, reported as [lower, upper].

Condition	What was tested	Shenlong CGA	Simple CGA	Euclidean 4×4	Key observation
5-object (n=8)	Translate, stack, scale, rotate, compose, multi-object, and hard spatial edits	6/8 (75%), 51 tok	8/8 (100%), 33 tok	7/8 (88%), 58 tok	Simple CGA is best on both reliability and cost.
Stress (n=6)	Irrational-angle rotation, triple composition, chained rotation, global scaling, relative stacking	6/6	6/6	5/6	Both CGA variants saturate this compositional set.
10-object (n=6)	Scene edits with larger context and multi-object references	6/6, 46 tok	6/6, 57 tok	6/6, 101 tok	Success ties; token cost differs strongly.
Spatial accuracy (n=18)	Expanded exact target placements (absolute coordinates + displacement constraints)	18/18 100.0% [82.4%, 100.0%] err 0.00	18/18 100.0% [82.4%, 100.0%] err 0.00	5/18 27.8% [12.5%, 50.9%] err 2.35	Both CGA variants saturate exact-placement success; Euclidean is substantially lower with larger geometric error.
100-object (n=10, 30-context)	Edits in procedurally generated large scene	9/10 (90%), 64 tok	10/10 (100%), 47 tok	10/10 (100%), 86 tok	Simple and Euclidean tie on success; Simple is much cheaper.

GA Meets LLM: Complete Results (33 Experiments)

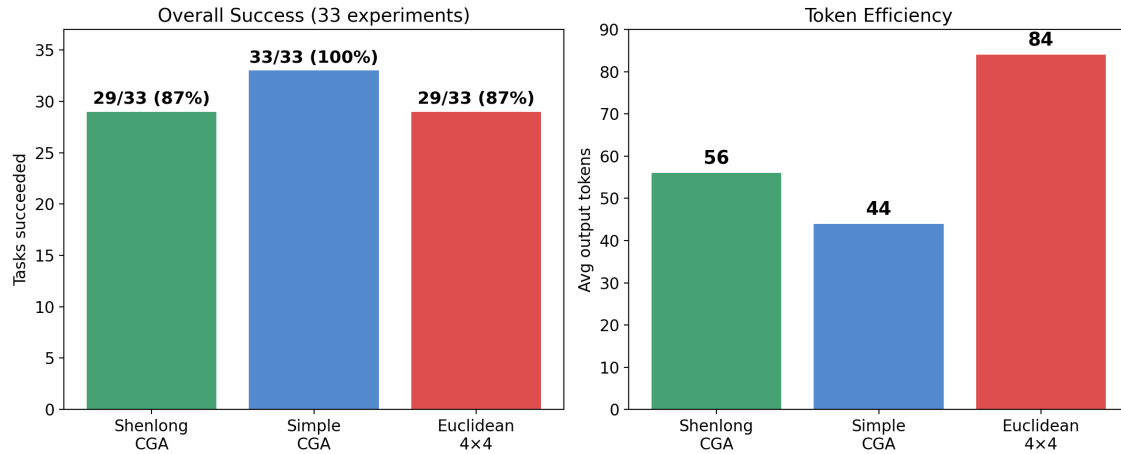


Fig. 7. Core-block mixed-endpoint overview across evaluated conditions.

To make the trade-off explicit in one consolidated view, Figure 8 plots an operational mixed-endpoint Pareto plane (success vs. completion tokens), while formal claims remain tied to endpoint-specific rows and hard-suite semantics.

This plot is included as a descriptive operational view rather than an inferential result.

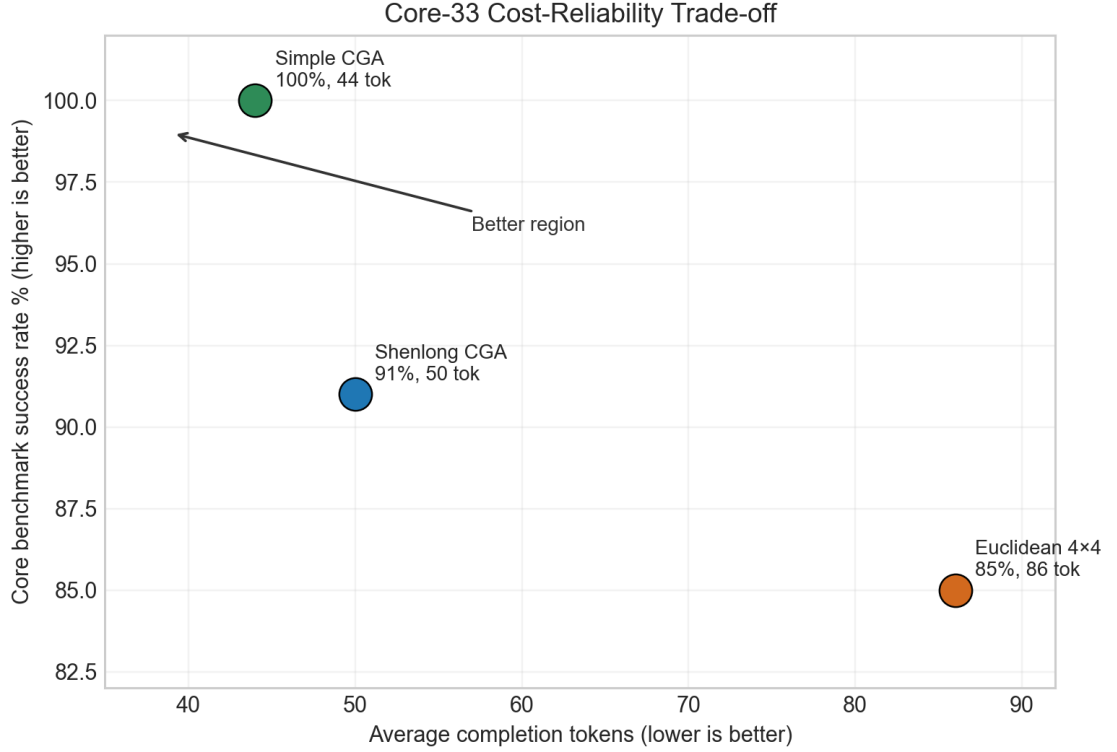


Fig. 8. Core-block mixed-endpoint success/token Pareto view.

5.4 Stability Under Repeated Runs

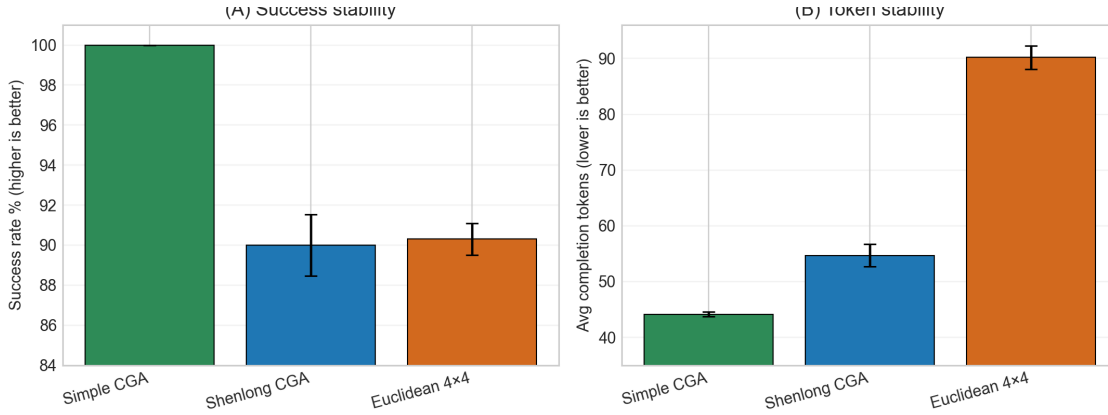
To separate stable effects from single-run variance, we repeat the full 33-task benchmark 10 times under fixed prompts and fixed default policy settings. Table 9 reports success and token uncertainty statistics, and Figure 9 visualizes the same means and confidence bands.

Table 9. Core-33 uncertainty summary across repeated runs (n=10 runs). Higher success mean is better. Lower SD/CI indicates higher stability. For token metrics (tok), lower values are better.

Approach	Success Mean	Success SD	Success 95% CI	Avg Tokens Mean	Avg Tokens SD	Avg Tokens 95% CI
Shenlong CGA	29.70	0.82	+/-0.51	54.70	3.24	+/-2.01
Simple CGA	33.00	0.00	+/-0.00	44.13	0.69	+/-0.43
Euclidean 4x4	29.80	0.42	+/-0.26	90.18	3.41	+/-2.11

The zero-SD result for Simple CGA is notable: across 10 independent runs, this method achieves identical task success with near-zero token variance. This pattern is consistent with higher run-to-run stability for the Simple CGA configuration in this benchmark.

Figure 9 is consistent with a strong and stable Simple CGA profile in this protocol.

Fig. 9. Core-33 repeated-run stability as mean $\pm$ 95% CI.

5.5 Semantic Validity Under Harder Grounding

Because parse-valid outputs can still diverge from geometric intent in harder settings, semantic checks are evaluated explicitly on two datasets: a semantic subset ($n=19$) and a harder language-grounding pack ($n=20$) with paraphrase, distractor, noisy-phrasing, and longer compositional instructions. Semantic outcomes are validated by geometry rules (surface contact, midpoint constraints, target displacements, scale factors, and absolute placement checks).

Calibration note: This section includes descriptive subsets ($n=19$, $n=20$) and the powered semantic suite ($n=100$ per method). The powered suite is used for confirmatory semantic inference, while smaller blocks provide descriptive support.

Table 10 summarizes these results.

Table 10. Parse-validity versus semantic-validity outcomes. Parse columns capture executable output structure; semantic columns capture geometric task satisfaction. Higher rates are better. For tokens, lower is better.

Dataset	Approach	Parse Success	Parse Rate	Semantic Success	Semantic Rate	Avg Tokens (parse-success rows)
Semantic subset ($n=19$)	Shenlong CGA	19/19	100.0%	9/19	47.4%	60.42
Semantic subset ($n=19$)	Simple CGA	19/19	100.0%	10/19	52.6%	41.37
Semantic subset ($n=19$)	Euclidean 4x4	18/19	94.7%	4/19	21.1%	81.06
Hard-pack ($n=20$)	Shenlong CGA	19/20	95.0%	9/20	45.0%	51.89
Hard-pack ($n=20$)	Simple CGA	20/20	100.0%	9/20	45.0%	39.20
Hard-pack ($n=20$)	Euclidean 4x4	20/20	100.0%	5/20	25.0%	64.35

Figure 10 provides the corresponding visual summary of the parse-semantic gap across the two evaluated datasets.

The parse-validity summary is therefore paired with explicit semantic correctness metrics.

To test retry sensitivity, hard-pack runs are pooled by policy ($k=1, 2, 3$) and compared in Table 11.

Figure 11 suggests that improving semantic grounding, rather than format consistency alone, is the more relevant improvement direction.

Pairwise Fisher exact tests on pooled repeats are summarized in Table 12.

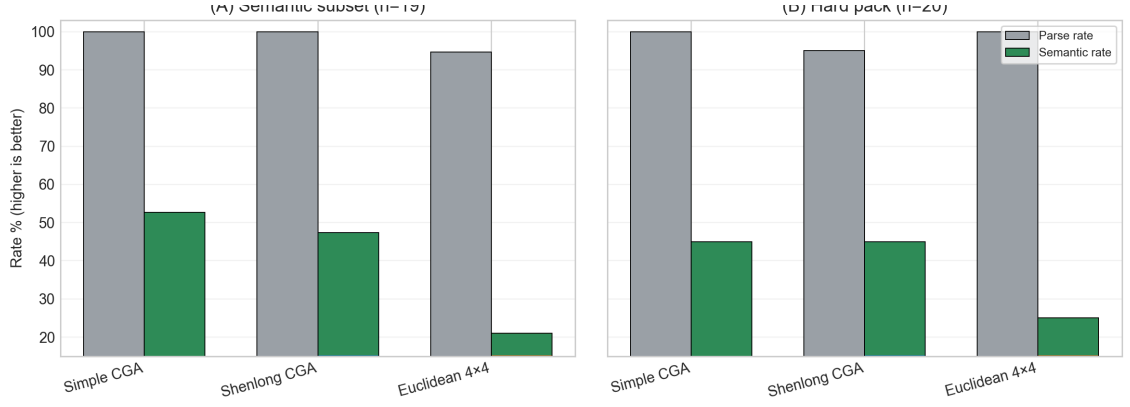


Fig. 10. Parse-semantic gap by dataset and method.

Table 11. Hard-pack pass@k sensitivity (pooled repeats). Higher parse/semantic rates are better.

Approach	Parse@1	Parse@2	Parse@3	Semantic@1	Semantic@2	Semantic@3
Shenlong CGA	95.0%	95.0%	95.0%	47.0%	45.0%	44.0%
Simple CGA	100.0%	100.0%	100.0%	45.0%	45.0%	45.0%
Euclidean 4x4	100.0%	100.0%	100.0%	25.0%	25.0%	25.0%

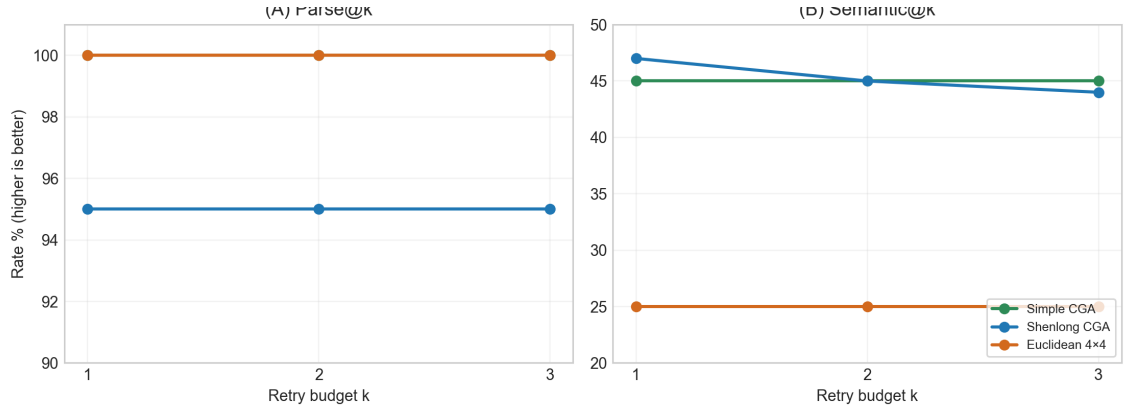


Fig. 11. Hard-pack pass@k sensitivity curves.

For completeness, Table 13 reports the compact SE3 control run ($n=20$) and the powered hard semantic suite ($n=100$ per method). Compact SE3 achieves complete parse coverage and semantic rates in the same range as the CGA variants; on token cost, it is the most compact representation in these runs.

In the powered suite ($n=100$), semantic pairwise contrasts versus Euclidean are confirmatory: Simple CGA vs Euclidean is +21 pp ($p=0.0028$), Shenlong vs Euclidean is +20 pp ($p=0.0044$), and Compact SE3 vs Euclidean is +18

Table 12. Hard-pack pairwise statistics (two-sided Fisher exact). Positive risk difference means Method A is higher; negative means Method A is lower. Significant p-values (<0.05) are shown in bold.

Comparison	Parse Risk Diff (pp)	Parse p-value	Semantic Risk Diff (pp)	Semantic p-value	Interpretation
Euclidean vs Shenlong	+5	0.0594	-22 to -19	0.0019 to 0.0072	Euclidean shows lower semantic rates despite high parse rates.
Euclidean vs Simple	+0	1.0000	-20	0.0047	Euclidean shows lower semantic rates than Simple CGA.
Shenlong vs Simple	-5	0.0594	-1 to +2	0.8872 to 1.0000	Hard-pack semantic rates are closely matched across CGA variants.

Table 13. Semantic follow-up datasets: compact non-CGA control and powered hard-suite results. The $n=100$ block supports confirmatory inference for compact-vs-Euclidean contrasts (see Table 14).

Dataset	Method	Parse Success	Parse Rate (Wilson 95% CI)	Semantic Success	Semantic Rate (Wilson 95% CI)	Avg Tokens (parse-success rows)
Hard-pack + Compact SE3 ($n=20$)	Compact SE3	20/20	100.0% [83.9%, 100.0%]	9/20	45.0% [25.8%, 65.8%]	23.65
Hard-pack + Compact SE3 ($n=20$)	Euclidean 4×4	20/20	100.0% [83.9%, 100.0%]	5/20	25.0% [11.2%, 46.9%]	64.15
Hard-pack + Compact SE3 ($n=20$)	Shenlong CGA	19/20	95.0% [76.4%, 99.1%]	9/20	45.0% [25.8%, 65.8%]	56.47
Hard-pack + Compact SE3 ($n=20$)	Simple CGA	20/20	100.0% [83.9%, 100.0%]	9/20	45.0% [25.8%, 65.8%]	36.85
Powered hard suite ($n=100$)	Compact SE3	100/100	100.0% [96.3%, 100.0%]	42/100	42.0% [32.8%, 51.8%]	21.45
Powered hard suite ($n=100$)	Euclidean 4×4	100/100	100.0% [96.3%, 100.0%]	24/100	24.0% [16.7%, 33.2%]	63.38
Powered hard suite ($n=100$)	Shenlong CGA	95/100	95.0% [88.8%, 97.8%]	44/100	44.0% [34.7%, 53.8%]	48.13
Powered hard suite ($n=100$)	Simple CGA	100/100	100.0% [96.3%, 100.0%]	45/100	45.0% [35.6%, 54.8%]	37.34

pp ($p=0.0103$). Simple CGA and Compact SE3 are statistically close on the same endpoint (+3 pp, $p=0.7755$), while Compact SE3 is the lowest-token method.

To make inference strength explicit, Table 14 reports semantic effect sizes with 95% CIs and p-values for the predefined contrasts.

Power-aware interpretation: in the powered suite, compact-vs-Euclidean semantic contrasts are both practically relevant (+18 to +21 pp) and statistically confirmatory ($p<0.05$), while the compact-vs-compact contrast is statistically close.

Adversarial-NL robustness is deferred to future work and is specified in Section 7 as a dedicated next-step evaluation item.

Table 14. Semantic effect sizes for predefined hard-suite contrasts. Positive risk difference favors the first method. Risk-difference CIs use normal approximation; relative-risk CIs use log-scale approximation; p-values are two-sided Fisher exact.

Dataset	Comparison	Semantic Success	Risk Difference (pp, 95% CI)	Relative Risk (95% CI)	Fisher p (2-sided)
Hard-pack (n=20)	Compact SE3 vs Euclidean	9/20 vs 5/20	+20.0 [-8.9, +48.9]	1.80 [0.73, 4.43]	0.3203
Hard-pack (n=20)	Simple CGA vs Euclidean	9/20 vs 5/20	+20.0 [-8.9, +48.9]	1.80 [0.73, 4.43]	0.3203
Hard-pack (n=20)	Shenlong vs Euclidean	9/20 vs 5/20	+20.0 [-8.9, +48.9]	1.80 [0.73, 4.43]	0.3203
Powered hard (n=100)	Compact SE3 vs Euclidean	42/100 vs 24/100	+18.0 [+5.2, +30.8]	1.75 [1.15, 2.66]	0.0103
Powered hard (n=100)	Simple CGA vs Euclidean	45/100 vs 24/100	+21.0 [+8.1, +33.9]	1.88 [1.24, 2.83]	0.0028
Powered hard (n=100)	Shenlong vs Euclidean	44/100 vs 24/100	+20.0 [+7.2, +32.8]	1.83 [1.21, 2.77]	0.0044
Powered hard (n=100)	Simple CGA vs Compact SE3	45/100 vs 42/100	+3.0 [-10.7, +16.7]	1.07 [0.78, 1.47]	0.7755

5.6 Multi-Model Replication

Table 15 shows the protocol-fixed multi-model check (gpt-4o-mini, gpt-4.1-mini) on hard-pack tasks (n=20 per model-method). Rates are directionally consistent across the two tested closed models: Compact SE3 and Simple CGA both exceed Euclidean on semantic success, and Compact SE3 is the lowest-token method in both models. External validity is improved within the closed-model family; open-model replication is the next expansion step and is explicitly deferred (see Section 7).

Table 15. Multi-model hard-pack replication (protocol-fixed, n=20 per model-method).

Model	Method	Parse Success	Parse Rate	Semantic Success	Semantic Rate	Avg Tokens (parse-success rows)
gpt-4o-mini	Compact SE3	20/20	100.0%	9/20	45.0%	21.25
gpt-4o-mini	Euclidean 4×4	20/20	100.0%	5/20	25.0%	62.95
gpt-4o-mini	Simple CGA	20/20	100.0%	9/20	45.0%	37.55
gpt-4.1-mini	Compact SE3	20/20	100.0%	11/20	55.0%	20.35
gpt-4.1-mini	Euclidean 4×4	20/20	100.0%	5/20	25.0%	63.75
gpt-4.1-mini	Simple CGA	20/20	100.0%	11/20	55.0%	34.00

Cross-model summary:

1. Holds in both tested closed models: Euclidean is at 25.0% hard-pack semantic success; Simple CGA and Compact SE3 are higher (45.0% on gpt-4o-mini, 55.0% on gpt-4.1-mini).
2. Holds in both tested closed models: Compact SE3 is the lowest-token method.
3. Not established by current data: stable separation between Simple CGA and Compact SE3, and transfer to open-weight model families.

Open-weight replication is deferred to future work and is specified in Section 7 as the next external-validity expansion step.

Table 16 summarizes key hard-suite trade-off metrics and pairwise significance against Euclidean.

Table 16. Cross-method hard-suite trade-off summary. Higher parse/semantic rates are better; lower tokens are better. p_{sem} vs Euclidean compares each method with Euclidean within the same dataset. The hard-pack $n=20$ rows are repeated from Table 10 for direct side-by-side comparison with the powered block.

Dataset	Method	Parse Rate (95% CI)	Semantic Rate (95% CI)	Avg Tokens	$p_{\{\text{sem}\}}$ vs Euclidean
Hard-pack ($n=20$)	Compact SE3	100.0% [83.9%, 100.0%]	45.0% [25.8%, 65.8%]	23.65	0.3203
Hard-pack ($n=20$)	Euclidean 4×4	100.0% [83.9%, 100.0%]	25.0% [11.2%, 46.9%]	64.15	-
Hard-pack ($n=20$)	Shenlong CGA	95.0% [76.4%, 99.1%]	45.0% [25.8%, 65.8%]	56.47	0.3203
Hard-pack ($n=20$)	Simple CGA	100.0% [83.9%, 100.0%]	45.0% [25.8%, 65.8%]	36.85	0.3203
Powered ($n=100$)	hard Compact SE3	100.0% [96.3%, 100.0%]	42.0% [32.8%, 51.8%]	21.45	0.0103
Powered ($n=100$)	hard Euclidean 4×4	100.0% [96.3%, 100.0%]	24.0% [16.7%, 33.2%]	63.38	-
Powered ($n=100$)	hard Shenlong CGA	95.0% [88.8%, 97.8%]	44.0% [34.7%, 53.8%]	48.13	0.0044
Powered ($n=100$)	hard Simple CGA	100.0% [96.3%, 100.0%]	45.0% [35.6%, 54.8%]	37.34	0.0028

The main inference from Tables 13–16 is that compact representations (Simple CGA and Compact SE3) are both substantially cheaper than Euclidean in token cost while remaining semantically stronger in the powered hard suite ($n=100$). At the same time, the semantic gap between the two compact variants is small ($p=0.7755$ at $n=100$), while sequence-fidelity still favors Simple CGA on the dedicated compositional benchmark (Table 7).

5.7 Interactive Latency and ENGAGE Pilot

We ran a powered full-loop latency protocol on the hard instruction set with $n=40$ rows per method (20 tasks $\times$ 2 runs; 120 calls total) under GPT-4o-mini and $\text{retries}=1$. Each call was timed end-to-end from instruction dispatch to render-ready state and decomposed into three components: LLM API time, parse/execute time, and render-ready preparation time. Parse validity was 100% for all three methods in this protocol, so the latency decomposition is not confounded by parse failures.

Table 17 summarizes these results.

The decomposition shows that latency is overwhelmingly API-bound for all methods (>99.8% of total mean time), while parse/execute and render-ready overheads remain sub-millisecond. This indicates that representation effects primarily act through token/API burden, not local geometric execution cost. The Euclidean condition includes one long API outlier (32.11 s), which inflates mean and SD; median total latencies still preserve the same ordering (Compact SE3: 0.81 s, Simple CGA: 1.19 s, Euclidean 4×4: 1.40 s).

The minimal within-subject user pilot (ENGAGE-focused) is deferred to immediate future work. The planned protocol is $n=5$ –10 participants, CGA-powered vs matrix-powered editing, primary endpoints task time and error rate, secondary endpoint usability preference/SUS.

Table 17. Full-loop latency decomposition for interactive editing ($n=40$ per method). Lower values are better. Mean $\pm$ SD and median [IQR] are both reported because the Euclidean condition contains a long-tail API outlier. Headroom is relative to the 100ms target; update rate is the reciprocal of mean total latency.

Method	Total Latency (s) mean $\pm$ SD	Total Latency (s) median [IQR]	API (s) mean $\pm$ SD	Parse+ Execute (ms mean)	Render- Ready (ms mean)	Headroom vs 100ms	Max Update Rate (Hz)
Compact SE3	0.94 $\pm$ 0.28	0.81 [0.38]	0.94 $\pm$ 0.28	0.69	0.20	+0.84 s	1.06
Simple	1.27 $\pm$ 0.38	1.19 [0.57]	1.27 $\pm$ 0.38	1.04	0.42	+1.17 s	0.79
CGA							
Euclidean 4 $\times$ 4	2.57 $\pm$ 4.91	1.40 [0.81]	2.57 $\pm$ 4.90	0.40	0.13	+2.47 s	0.39

5.8 Ablation Diagnostics

To isolate confounds, we run repeated ablations on a fixed 10-task set under two policies (pass@1, pass@2) and four conditions (Simple compact, Simple verbose, Shenlong, Euclidean), yielding 50 evaluated rows per condition-policy. Full tables are preserved in Appendix A (Tables 25–26); this section summarizes the key diagnostic findings.

Table 18 summarizes these results.

Table 18. Repeated ablation summary (parse-focused; summary table). Parse rate is shown with Wilson CI (higher is better). Completion tokens and latency are cost metrics (lower is better). Shenlong’s lower parse rate relative to the Simple variants is partly associated with max-token budget effects under standard settings. Retry policy modestly shifts cost and Shenlong parse rates, while compact/verbose Simple variants remain parse-stable.

Policy	Condition	Parse Rate (Wilson CI)	Avg Completion Tokens	Avg Latency (s)
pass@1	Simple CGA (compact)	100.0% [92.9%, 100.0%]	36.72	1.29
pass@1	Simple CGA (verbose)	100.0% [92.9%, 100.0%]	34.40	1.32
pass@1	Shenlong CGA	80.0% [67.0%, 88.8%]	46.70	1.53
pass@1	Euclidean 4 $\times$ 4	100.0% [92.9%, 100.0%]	71.58	1.89
pass@2	Simple CGA (compact)	100.0% [92.9%, 100.0%]	41.02	1.44
pass@2	Simple CGA (verbose)	100.0% [92.9%, 100.0%]	33.54	1.45
pass@2	Shenlong CGA	82.0% [69.2%, 90.2%]	46.34	1.68
pass@2	Euclidean 4 $\times$ 4	100.0% [92.9%, 100.0%]	74.50	1.79

The ablation evidence supports two conservative claims: (i) Shenlong’s parse-rate headroom stays under modest retry increases, and this deficit should be attributed to its prompt-length budget interaction rather than to any fundamental CGA representational weakness; (ii) representation-level and prompt-level effects interact, so token count alone should not be treated as a single explanatory variable. Full pairwise statistics are in Appendix A (Table 26).

Figure 12 shows the Simple variants on a stronger parse-rate/cost frontier, while Euclidean shifts rightward in completion cost.

Interpretation limits: several subsets are modest in size (semantic subset $n=19$, hard-pack $n=20$), and policy choice (pass@1 vs pass@k) affects parse-oriented outcomes (Tables 11 and 18). Accordingly, rates are reported as protocol-conditional evidence under the tested model family, prompts, and retry settings, rather than universal performance constants.

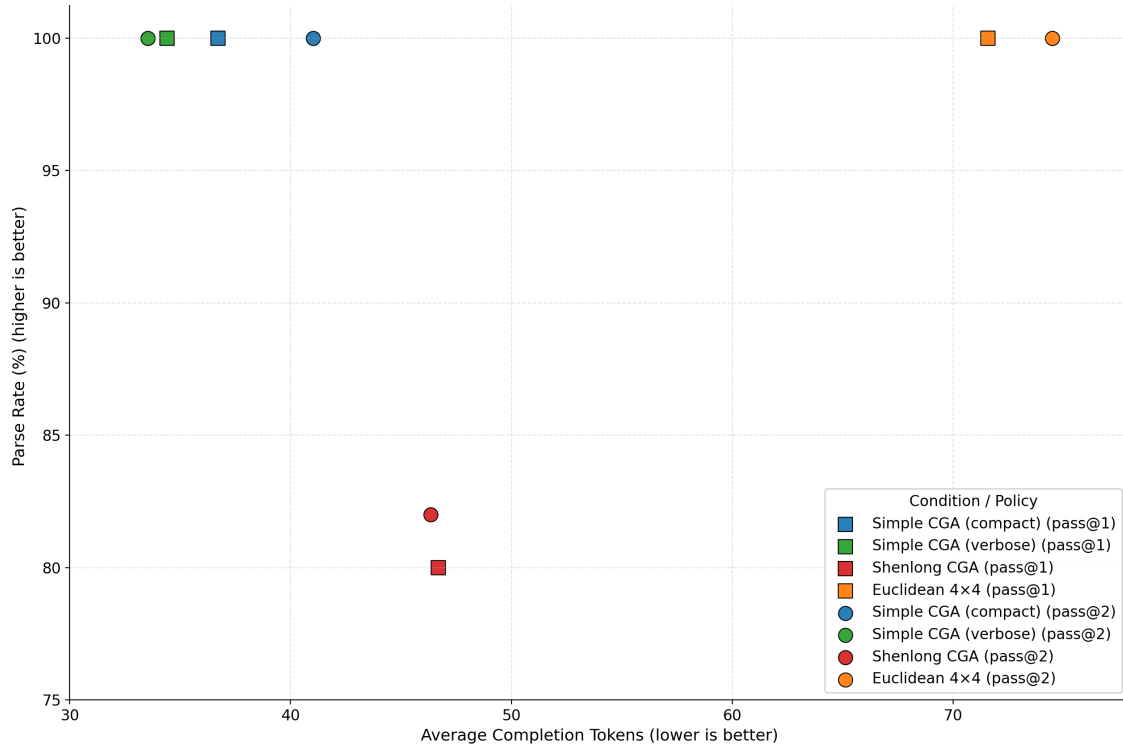


Fig. 12. Ablation parse-rate vs completion-token trade-off.

5.9 Reproducibility and Supplementary Material

To keep the main narrative readable while preserving traceability, the following supporting assets are moved to Appendix A:

- protocol snapshot (Table 19);
- detailed per-task core tables and supplementary figures (Tables 20–23, Figures 13–15);
- compact visual summaries are shown in main text (Figures 7–12), while full supporting numeric rows remain in Appendix A tables;
- follow-up datasets (hard-pack + Compact SE3, multi-model hard-pack, expanded hard-suite uncertainty) are included in the supplementary materials;
- hard-pack retry-aware cost table (Table 24);
- single-run ablation and full repeated-ablation pairwise matrix (Tables 25–26);
- consolidated supplementary completeness tables (Tables 27–28).

6 Discussion

6.1 Sequence-Fidelity Evidence for Algebraic Expressiveness

The sequence-fidelity result (Table 7, Section 5.2) shows a +7.5 pp advantage for Simple CGA over Compact SE3 ($p=0.016$) under parse saturation for both methods, with a modest token gap (21 tokens). Because both methods are compact and both parse at 100%, this contrast isolates ordered-chain behavior rather than parseability.

A plausible account is that CGA motor composition encodes a chain as a single algebraic object, whereas Compact SE3 uses list-position semantics across typed JSON operations. The data are consistent with this hypothesis, but they do not by themselves establish a causal mechanism. Latency should be read as a separate axis in Table 17: Compact SE3 is faster on mean latency, while Simple CGA retains the sequence-fidelity advantage.

6.2 Semantic Trends for Compact Interfaces

Hard-pack and semantic checks show that parse success is only a partial proxy for geometric task success (Tables 10 and 11), so syntax-valid outputs and geometry-valid outcomes should be reported separately.

Under that criterion, the powered hard-suite results ($n=100$) show that all three compact interfaces (Simple CGA, Shenlong CGA, and the Compact SE3 control) outperform the Euclidean baseline on semantic success (Table 14, $p<0.05$ in all three compact-vs-Euclidean contrasts). The Simple CGA vs Compact SE3 semantic contrast remains statistically close ($p=0.7755$), so the supported aggregate claim is compact-interface advantage over the tested Euclidean baseline, with a Simple-CGA advantage confined to the sequence-fidelity endpoint.

6.3 Latency and Interactive Viability

The token-efficiency story, when translated to wall-clock latency (Table 3), has direct relevance for immersive NL-editing interfaces. The full-loop measurements in Section 5.7 show Compact SE3 as fastest (0.94 ± 0.28 s), followed by Simple CGA (1.27 ± 0.38 s), with Euclidean 4×4 slower on mean latency (2.57 ± 4.91 s). Relative to a 100ms target, the measured headroom is +0.84 s, +1.17 s, and +2.47 s, respectively.

All tested methods are currently above the 100ms sub-interactive threshold. The gap is dominated by LLM API latency rather than representation overhead; architectural work on streaming execution, caching, or local inference would need to close an order of magnitude. In this protocol, API time contributes more than 99.8% of total latency for every method, while parse/execute and render-ready stages are sub-millisecond. The representation choice still matters because it modulates token/API burden: both compact interfaces (Compact SE3, Simple CGA) retain clear latency advantages over the verbose Euclidean baseline, and these differences accumulate over multi-step edits.

6.4 Scope, Validity, and Practical Implications

The claims are valid for the evaluated system: GPT-4o-mini / GPT-4.1-mini, the current template-execution pipeline, primitive-shape scenes, and the tested task distributions. Immediate next-step validation includes transfer tests to additional model families, open-world 3D assets, and more unconstrained instructions. The matched closed-model check (Table 15) supports directional consistency within the GPT family and provides a concrete launch point for broader cross-provider transfer studies.

Within those bounds, the practical implication is clear: choosing a compact, closed-form geometric interface is a high-leverage design decision for LLM-to-geometry systems. CGA is particularly attractive when unified algebraic

composition is required — specifically, when ordered instruction chains and motor composition semantics are desirable — and the current evidence motivates broader comparative studies across compact symbolic encodings.

6.5 Threats to Validity

Three named threats are identified proactively:

Threat 1: Training-data contamination. GPT-family models may have encountered CGA syntax during pre-training (e.g., from academic code repositories or tutorial pages for the `clifford` or `kingdon` Python libraries). If the model has memorized CGA expression patterns, its "generation" of CGA motors may partially reflect recall rather than compositional reasoning, inflating CGA's apparent advantage over less well-represented formalisms. Mitigation: open-weight model replication (Section 7) on models with more transparent training corpora would partially address this threat. The Compact SE3 control baseline, being a bespoke JSON schema unlikely to appear in pre-training data, provides some protection against this confound in the semantic suite; the sequence-fidelity result (Section 5.2) uses the same Compact SE3 control for a direct compact-vs-compact comparison that is less susceptible to recall-based inflation.

Threat 2: Compact SE3 underspecification risk. Comparative studies can under-specify compact non-CGA baselines, making fairness difficult to assess. This paper mitigates that risk by providing a full Compact SE3 schema and execution semantics in Section 4.3. A remaining limitation is that per-task schema interpretation traces are not reported in the main text.

Threat 3: Shenlong max-tokens budget artifact. The Shenlong CGA variant's lower parse rates (80–82% in ablation, Table 17) are partly attributable to a max-tokens budget interaction: its verbose chain-of-thought prompt generates 200–500 additional reasoning tokens that can push total output past the token limit, causing truncated JSON. This makes Shenlong's apparent disadvantage relative to Simple CGA a partially unfair comparison at standard token budgets. Reporting Shenlong as a legacy engineering reference point rather than a primary competitor (as done in Section 4.2) is the editorial response; a fairness run with a larger output budget would allow a cleaner statement.

7 Conclusion

This paper evaluates whether output representation changes the reliability and compositional fidelity of LLM-driven 3D scene editing. Our strongest result is confirmatory: under a sequence-stress protocol designed to isolate ordered-chain faithfulness ($n=120$ outputs per method), Simple CGA preserves exact operation sequences more reliably than a Compact SE3 control (97.5% vs 90.0%, $p=0.016$) while using fewer completion tokens (112.6 vs 133.6). This result holds under parse saturation (both methods at 100% parse rate), and is consistent with the hypothesis that algebraic expression form — not compactness alone — is associated with higher compositional faithfulness.

A second result is now confirmatory on the powered semantic suite ($n=100$ per method): compact representations (Simple CGA 45.0%, Compact SE3 42.0%, Shenlong 44.0%) exceed the Euclidean 4×4 baseline (24.0%), with effect sizes of +21 pp ($p=0.0028$) for Simple CGA vs Euclidean, +18 pp ($p=0.0103$) for Compact SE3 vs Euclidean, and +20 pp ($p=0.0044$) for Shenlong vs Euclidean. Observed effect sizes are practically relevant, while the Simple CGA vs Compact SE3 semantic difference is statistically close (+3 pp, $p=0.7755$). The protocol is fixed and the compact-vs-Euclidean direction is consistent across two closed model families.

A third contribution is methodological. The consistent gap between parse success and semantic success across all tested methods and datasets indicates that syntax-valid outputs are a necessary but insufficient proxy for geometric correctness. Future evaluation protocols for LLM-to-geometry systems should explicitly separate these two criteria.

Immediate future work should prioritise four directions. First, run an adversarial natural-language robustness subset (10-20 instructions authored without representation hints) under the fixed protocol and report method-wise semantic rates as a dedicated robustness endpoint. Second, apply grammar-constrained CGA decoding — using structured generation frameworks such as Outlines or Guidance to enforce the CGA expression grammar by construction — to eliminate residual parse failures and isolate semantic grounding as the remaining frontier. Third, perform open-weight model replication (e.g., Llama-3/Mistral families) under matched settings to test whether the compact-vs-verbose trend transfers beyond the GPT model family and to reduce training-data-contamination concerns. Fourth, add closed-loop visual verification — coupling the execution engine to a rendering oracle that scores geometric satisfaction automatically — as a scalable path to larger semantic suites for confirmatory inference. Together these steps outline a path from the current evidence to more robust, model-general conclusions about symbolic interface design for LLM-to-geometry pipelines.

8 Acknowledgments

This work was partially supported by the Innosuisse OMEN-E project and by the European Union's research and innovation programs. The authors thank the Kingdon library developers for the open-source Geometric Algebra implementation [19] that made this work possible.

9 Disclosure of Interests

The authors declare the following interests: G. Papagiannakis is the founder and CEO of ORamaVR SA, which develops the MAGES SDK platform referenced in this paper.

References

- [1] Andreas Aristidou and Joan Lasenby. 2011. Inverse Kinematics Solutions Using Conformal Geometric Algebra. In *Guide to Geometric Algebra in Practice*. Springer, London, 47–62.
- [2] Armen Avetisyan, Chen Xie, Henry Howard-Jenkins, Tzu-Yin Yang, Sarah Aroudj, Sayan Patra, Fan Zhang, Daniel P. Frost, Luke Holland, Colin Orme, Jakob J. Engel, Eric Miller, Richard A. Newcombe, and Vassileios Balntas. 2024. SceneScript: Reconstructing Scenes With An Autoregressive Structured Language Model. In *Computer Vision – ECCV 2024*, Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.). Lecture Notes in Computer Science, Vol. 15119. Springer, 247–263. doi:10.1007/978-3-031-73030-6_14
- [3] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting Is Programming: A Query Language for Large Language Models. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 1946–1969. doi:10.1145/3591300
- [4] Chris Doran and Anthony Lasenby. 2003. *Geometric Algebra for Physicists*. Cambridge University Press, Cambridge.
- [5] Leo Dorst, Daniel Fontijne, and Steven Mann. 2007. *Geometric Algebra for Computer Science: An Object-Oriented Approach to Geometry*. Morgan Kaufmann, Burlington.
- [6] Artur d'Ávila Garcez and Luis C. Lamb. 2023. Neurosymbolic AI: The 3rd Wave. *Artificial Intelligence Review* 56 (2023), 12387–12406. doi:10.1007/s10462-023-10448-w
- [7] Guidance-AI. [n. d.]. Guidance - A Guidance Language for Controlling Large Language Models. <https://github.com/guidance-ai/guidance>. Accessed 20 Apr 2026.
- [8] David Hestenes and Garret Sobczyk. 1984. *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. D. Reidel Publishing, Dordrecht.
- [9] Dietmar Hildenbrand. 2013. *Foundations of Geometric Algebra Computing*. Springer, Berlin. doi:10.1007/978-3-642-31794-1

- [10] Wenlong Huang, Chen Wang, Ruihan Zhang, et al. 2023. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. In *Conference on Robot Learning (CoRL) (PMLR, Vol. 229)*.
- [11] Panagiotis Kolyvakis, Manos Kamarianakis, and George Papagiannakis. 2025. Geometric Algebra Meets Large Language Models: Instruction-Based Transformations of Separate Meshes in 3D, Interactive and Controllable Scenes. In *2025 IEEE International Symposium on Emerging Metaverse (ISEMV)*. 36–45.
- [12] Jacky Liang, Wenlong Huang, Fei Xia, et al. 2023. Code as Policies: Language Model Programs for Embodied Control. In *IEEE International Conference on Robotics and Automation (ICRA)*. 9493–9500.
- [13] Wenxuan Liu, Yifeng Du, Tucker Hermans, Sonia Chernova, and Chris Paxton. 2023. StructDiffusion: Language-Guided Creation of Physically-Valid Structures using Unseen Objects. In *Proceedings of Robotics: Science and Systems (RSS)*. doi:10.15607/RSS.2023.XIX.031
- [14] Yiming Mao, Jun Zhong, Chao Fang, Jie Zheng, Rui Tang, Hao Zhu, Ping Tan, and Ziqi Zhou. 2025. SpatialLM: Training Large Language Models for Structured Indoor Modeling. arXiv:2506.07491 [cs.CV] doi:10.48550/arXiv.2506.07491 arXiv preprint.
- [15] George Papagiannakis, Manos Kamarianakis, Antonis Protopsaltis, Dimitris Angelis, and Paul Zikas. 2023. Project Elements: A Computational Entity-component-system in a Scene-graph Pythonic Framework, for a Neural, Geometric Computer Graphics Curriculum. In *Eurographics 2023 - Education Papers*, Alejandra Magana and Jiri Zara (Eds.). The Eurographics Association. doi:10.2312/eged.20231015
- [16] George Papagiannakis, Panagiotis Papanikolaou, Eirini Greassidou, and Panos Trahanias. 2014. glGA: An OpenGL Geometric Application Framework for a Modern, Shader-based Computer Graphics Curriculum. In *Eurographics 2014 - Education Papers*.
- [17] George Papagiannakis, Paul Zikas, Nikolas Lydatakis, et al. 2024. MAGES 4.0: Accelerating the World’s Transition to VR Training and Assessment. *IEEE Computer Graphics and Applications* 44, 1 (2024), 46–56.
- [18] Krishan Rana, Jesse Haviland, Sarthak Garg, et al. 2023. SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Robot Task Planning. In *Conference on Robot Learning (CoRL) (PMLR, Vol. 229)*.
- [19] Martin Roelfs. 2025. The Willing Kingdon Clifford Algebra Library. arXiv:2503.10451 [cs.MS] <https://arxiv.org/abs/2503.10451>
- [20] John Vince. 2008. *Geometric Algebra for Computer Graphics*. Springer, London. doi:10.1007/978-1-84628-997-2
- [21] Brandon T. Willard and Rémi Louf. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702 [cs.CL] doi:10.48550/arXiv.2307.09702

10 Appendix A: Supplementary Results and Validation Tables

This appendix preserves detailed outputs that were removed from the main Section 5 for readability. No experimental results are discarded.

10.1 Protocol Snapshot

Table 19. Protocol snapshot and fixed settings used by the main benchmarks, including model selection, prompt lengths, retry policy, and token budgets for the core benchmark, powered semantic suite, and powered latency protocol.

Field	Value
Snapshot timestamp	2026-04-16 19:12:37
Model	gpt-4o-mini
Prompt length (Simple/Shenlong/Euclidean/Compact SE3)	721 / 963 / 435 / 588 characters
Temperature schedule	0.1 + 0.05 * attempt
Retry policy (defaults)	run_one retries = 2; stress max shots = 3
Max tokens per core-33 benchmark block	5-object: 500/300/400, stress: 500/500/500, 10-object: 500/400/500, accuracy: 300/300/300, 100-object: 600/600/600 (Shenlong/Simple/Euclidean)
Max tokens (powered semantic suite, Section 5.5)	model: gpt-4o-mini; n=100 per method (20 tasks × 5 variants); retries=1; max_tokens: 600 (Simple/Shenlong/Euclidean), 500 (Compact SE3)
Powered latency protocol settings (Section 5.7)	model: gpt-4o-mini; n=40 per method (20 tasks × 2 runs); retries=1; max_tokens: 600 (Simple/Euclidean), 500 (Compact SE3)

Table 20. Task-level 5-object benchmark outcomes across methods, with per-task completion-token counts. OK denotes parse and semantic success; FAIL denotes parse failure or semantic failure.

Task Type	Shenlong	Tokens	Simple CGA	Tokens	Euclidean	Tokens
Simple translate	OK	127	OK	25	OK	41
Stacking	OK	33	OK	29	OK	45
Scaling	OK	15	OK	12	FAIL	0
Rotation	FAIL	0	OK	20	OK	41
T+R+D compose	OK	49	OK	47	OK	55
Stack+scale	OK	38	OK	34	OK	41
Multi-object	OK	45	OK	77	OK	123
Hard spatial	FAIL	0	OK	22	OK	57
Total	6/8 (75%)	avg 51	8/8 (100%)	avg 33	7/8 (88%)	avg 58

Table 21. Task-level stress-test outcomes under pass@3, showing per-task success/failure by method across compositional and robustness-oriented instruction types.

Task Type	Shenlong	Simple CGA	Euclidean
Irrational angle	OK	OK	OK
Triple compose	OK	OK	OK
5-obj edit	OK	OK	OK
Chained rotation	OK	OK	OK
Scale all	OK	OK	FAIL
Stack+relative	OK	OK	OK
Total	6/6	6/6	5/6

Table 22. Expanded spatial-accuracy suite (n=18) with inferential summary. Exact success uses threshold error <0.5 ; Wilson 95% CIs are shown as [lower, upper]. Lower errors are better.

Method	Exact Success	Exact Rate (Wilson 95% CI)	Mean Error	Median Error	Avg Tokens (parse-success rows)
Shenlong CGA	18/18	100.0% [82.4%, 100.0%]	0.00	0.00	44.56
Simple CGA	18/18	100.0% [82.4%, 100.0%]	0.00	0.00	29.50
Euclidean 4×4	5/18	27.8% [12.5%, 50.9%]	2.35	3.30	47.11

10.2 Detailed Core Benchmark Tables

Pairwise exact-success tests (two-sided Fisher) on this suite: Euclidean vs Shenlong $p < 1e-5$; Euclidean vs Simple CGA $p < 1e-5$; Shenlong vs Simple CGA $p = 1.0$.

Table 23. 100-object benchmark summary under the 30-object context setting, reporting method-wise success and average completion-token cost.

Approach	Success	Avg Tokens
Shenlong CGA	9/10 (90%)	64
Simple CGA	10/10 (100%)	47
Euclidean 4×4	10/10 (100%)	86

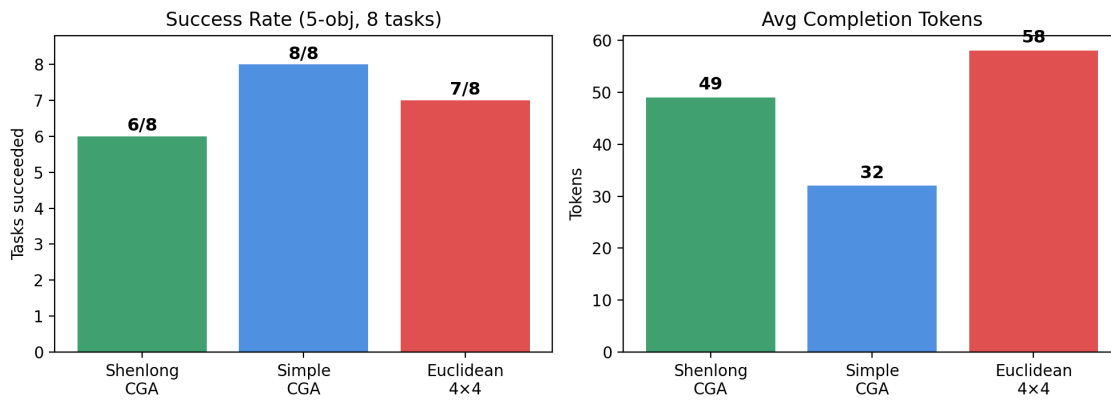
5-Object Scene: Three-Way Comparison

Fig. 13. 5-object scene comparison.

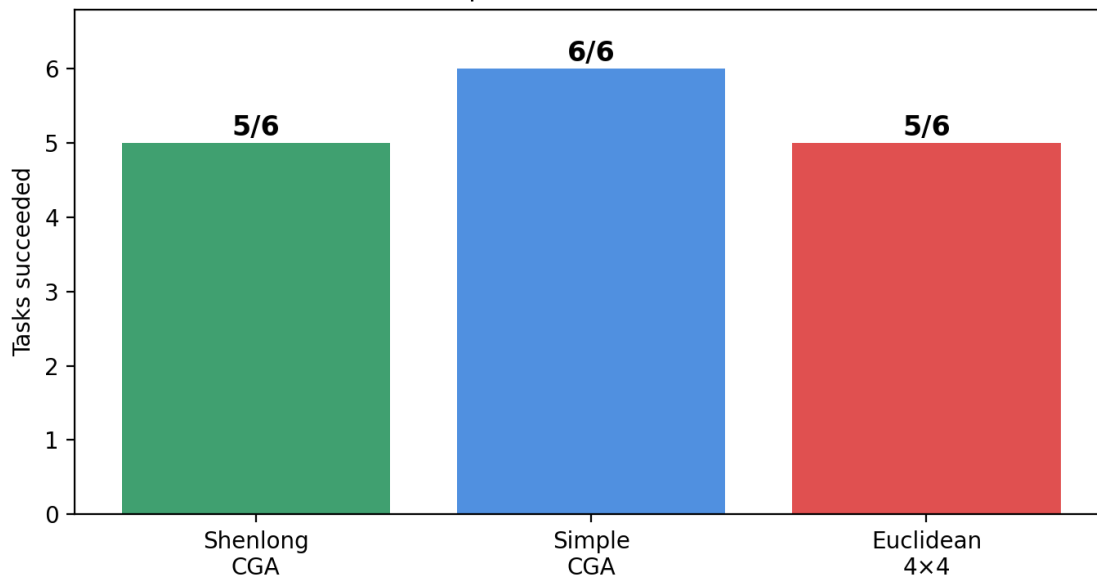
Stress Test: Compositional Tasks (6 tasks, 3 retries)

Fig. 14. Stress-test comparison.

Supplementary full-context run (all 100 object descriptions): Shenlong 9/10 (avg 64), Simple CGA 10/10 (avg 50), Euclidean 10/10 (avg 85).

Figure 13 compares 5-object outcomes; higher success and lower tokens are preferable.

Figure 14 summarizes stress-test behavior under the same instruction structure.

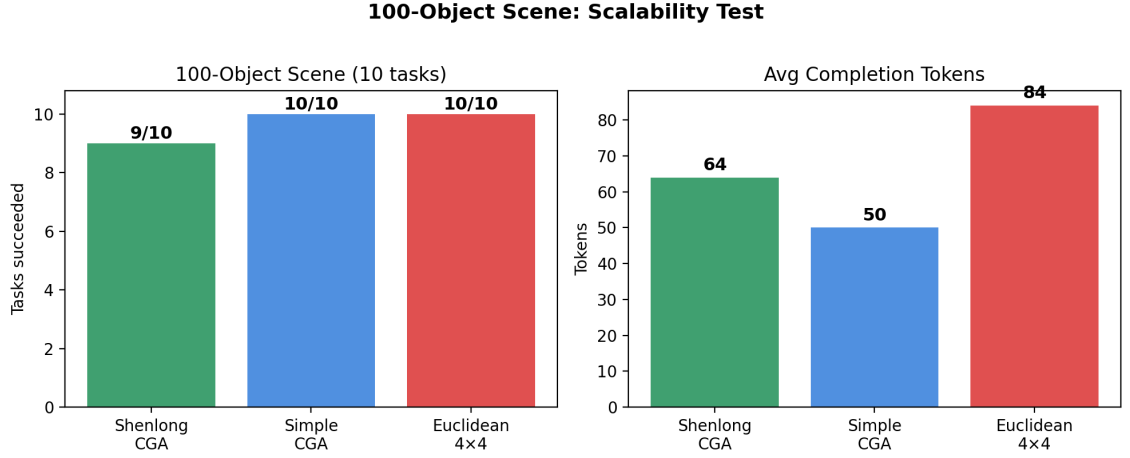


Fig. 15. 100-object benchmark comparison.

Figure 15 compares 100-object benchmark outcomes, where lower token cost is better at matched success.

10.3 Cost, Ablation, and Verification Support Tables

Table 24. Hard-pack retry-aware cost and latency by method and retry budget (k), reporting average completion tokens (success rows), average total tokens across all attempts, and average per-task latency.

Retries (k)	Approach	Avg Completion Tokens (success rows)	Avg Total Tokens (all attempts)	Avg Latency per Task (s)
1	Shenlong CGA	52.48	601.26	1.40
1	Simple CGA	37.99	513.09	1.24
1	Euclidean 4×4	64.24	456.34	1.57
2	Shenlong CGA	54.34	633.32	2.29
2	Simple CGA	38.35	513.45	1.27
2	Euclidean 4×4	63.77	455.87	1.56
3	Shenlong CGA	50.65	660.12	1.55
3	Simple CGA	38.00	513.10	1.30
3	Euclidean 4×4	64.62	456.72	1.58

11 Appendix B: Implementation Details

11.1 CGA Primitives Implementation

The CGA primitives are implemented using the Kingdon library for Python [19]. The algebra is constructed as $\text{Algebra}(4, 1)$, yielding the 32-dimensional $\text{Cl}(4,1)$ space. Key implementation functions:

```
cga = Algebra(4, 1)
e1, e2, e3 = cga.blades['e1'], cga.blades['e2'], cga.blades['e3']
```

Table 25. Single-run ablation summary by policy and condition, reporting tasks attempted, successes, success rates, and average completion tokens for successful rows.

Policy	Condition	Tasks	Success	Success Rate	Avg Tokens (success rows)
pass@1	Simple CGA (compact)	10	10	100.0%	42.40
pass@1	Simple CGA (verbose)	10	10	100.0%	35.50
pass@1	Shenlong CGA	10	8	80.0%	49.25
pass@1	Euclidean 4×4	10	10	100.0%	74.00
pass@2	Simple CGA (compact)	10	10	100.0%	60.30
pass@2	Simple CGA (verbose)	10	10	100.0%	35.40
pass@2	Shenlong CGA	10	8	80.0%	51.25
pass@2	Euclidean 4×4	10	10	100.0%	71.10

Table 26. Full repeated-ablation pairwise matrix across pass@1 and pass@2, including rate contrasts, risk difference, relative risk, odds ratio, and two-sided Fisher p-values.

Policy	Retries (k)	Condition A	Condition B	Rate A	Rate B	Risk Difference	Relative Risk	Odds Ratio	Fisher p-value (2-sided)
pass@1	1	Simple (compact)	CGA Simple (verbose)	100.0%	100.0%	+0.0 pp	1.000	1.000	1.0000
pass@1	1	Simple (compact)	CGA Shenlong CGA	100.0%	80.0%	+20.0 pp	1.247	26.185	0.0012
pass@1	1	Simple (compact)	CGA Euclidean 4×4	100.0%	100.0%	+0.0 pp	1.000	1.000	1.0000
pass@1	1	Simple (verbose)	CGA Shenlong CGA	100.0%	80.0%	+20.0 pp	1.247	26.185	0.0012
pass@1	1	Simple (verbose)	CGA Euclidean 4×4	100.0%	100.0%	+0.0 pp	1.000	1.000	1.0000
pass@1	1	Shenlong CGA	CGA Euclidean 4×4	80.0%	100.0%	-20.0 pp	0.802	0.038	0.0012
pass@2	2	Simple (compact)	CGA Simple (verbose)	100.0%	100.0%	+0.0 pp	1.000	1.000	1.0000
pass@2	2	Simple (compact)	CGA Shenlong CGA	100.0%	82.0%	+18.0 pp	1.217	23.120	0.0026
pass@2	2	Simple (compact)	CGA Euclidean 4×4	100.0%	100.0%	+0.0 pp	1.000	1.000	1.0000
pass@2	2	Simple (verbose)	CGA Shenlong CGA	100.0%	82.0%	+18.0 pp	1.217	23.120	0.0026
pass@2	2	Simple (verbose)	CGA Euclidean 4×4	100.0%	100.0%	+0.0 pp	1.000	1.000	1.0000
pass@2	2	Shenlong CGA	CGA Euclidean 4×4	82.0%	100.0%	-18.0 pp	0.822	0.043	0.0026

```

e4, e5 = cga.blades['e4'], cga.blades['e5']
no = 0.5 * (e5 - e4)    # null origin
ni = e4 + e5            # null infinity

def T(vec):
    """Translation motor: T(t) = 1 - 0.5 * t * ni"""
    return ONE - 0.5 * vec * ni

```

Table 27. Consolidated supplementary completeness checklist confirming the presence of protocol, uncertainty, semantic, ablation, and pairwise-reporting artifacts in the supplementary material.

Checklist Item	Status
Protocol snapshot present	Yes
Core pass-policy summary present	Yes
Core uncertainty summary present	Yes
Semantic subset summary present	Yes
Ablation summary present	Yes
Hard-pack summary present	Yes
Hard-pack rows present	Yes
Pairwise rows written	6

Table 28. Single-run hard-pack pairwise cross-check (n=20 per method), listing parse and semantic pairwise contrasts with rate differences, relative risk, odds ratio, and two-sided Fisher p-values.

Dataset	Metric	Method A	Method B	Rate A	Rate B	Risk Difference	Relative Risk	Odds Ratio	Fisher p-value (2-sided)
hardpack latest	parse	Euclidean	Shenlong	100.0%	95.0%	+5.0 pp	1.051	3.154	1.0000
hardpack latest	parse	Euclidean	Simple CGA	100.0%	100.0%	+0.0 pp	1.000	1.000	1.0000
hardpack latest	parse	Shenlong	Simple CGA	95.0%	100.0%	-5.0 pp	0.951	0.317	1.0000
hardpack latest	semantic	Euclidean	Shenlong	25.0%	45.0%	-20.0 pp	0.579	0.430	0.3203
hardpack latest	semantic	Euclidean	Simple CGA	25.0%	45.0%	-20.0 pp	0.579	0.430	0.3203
hardpack latest	semantic	Shenlong	Simple CGA	45.0%	45.0%	+0.0 pp	1.000	1.000	1.0000

```
def R(angle, u, v):
    """Rotation motor:  $R(\theta, u, v) = \cos(\theta/2) - \sin(\theta/2) * (u \wedge v)$ """
    return math.cos(angle/2)*ONE - math.sin(angle/2)*(u ^ v)

def D(s):
    """Dilation motor:  $D(s) = \cosh(\ln(s)/2) + \sinh(\ln(s)/2) * (no \wedge ni)$ """
    E = no ^ ni
    g = math.log(s) / 2
    return math.cosh(g)*ONE + math.sinh(g)*E
```

11.2 Verification Tests

All CGA primitives pass verification against known results:

- **Translation composition:** $T(2, 1, 0) \cdot T(1, 0, 3)$ on origin yields $(3, 1, 3)$ [OK]
- **Rotation isometry:** Distance between two points preserved under rotation [OK]

- **Motor composition:** $R(90^\circ, XY)$ then $T(5, 0, 0)$ on $(1, 0, 0)$ yields $(5, 1, 0)$ [OK]
- **Dilation:** $D(3)$ on $(2, 0, 0)$ yields $(6, 0, 0)$ [OK]

11.3 System Prompt (Simple CGA)

You are a 3D scene editor outputting CGA operations as JSON.

AXES: $e_1=X(\text{right})$, $e_2=Y(\text{up})$, $e_3=Z(\text{towards viewer})$

OPS: $T(x*e_1+y*e_2+z*e_3)=\text{translate}$, $R(\text{angle_rad}, e_i, e_j)=\text{rotate}$, $D(s)=\text{scale}$

$R()$ takes radians and TWO BASIS VECTORS: $R(\text{np.pi}/2, e_1, e_2)$.

NEVER pass numbers for the plane axes.

$T()$ takes a DISPLACEMENT vector (delta), NOT an absolute target position.

Compose: $T(\dots)*R(\dots) = \text{rotate then translate}$. Use np.pi , math.sqrt .

"next to"=surfaces touch (sum of sizes apart). "on top"=bottom on top surface.

"between"=midpoint of the two referenced objects.

OUTPUT: ONLY JSON. Keys=object names, Values=CGA expression strings.

Example: move sphere from $[1,0,0]$ to $[4,2,0]$ -> $\{\text{"Sphere": "T(3*e1+2*e2)"}$

11.4 Detailed Spatial-Edit Derivations

Case 1: "Move the red sphere next to the blue cube, to its left side."

Step 1 – Parse: Scan for color keywords. "red" at position 9 → RedSphere (mover), "blue" at position 38 → BlueCube (target). Keyword "left" → use "next to" logic.

Step 2 – Compute displacement:

- RedSphere: center= $[0,0,0]$, size=1.0
- BlueCube: center= $[4,0,0]$, size=1.0, min= $[3,-1,-1]$

$$d_x = x_{\min}^{\text{target}} - r_{\text{mover}} - x_{\text{mover}} = 3 - 1.0 - 0 = 2.0, \quad d_y = 0, \quad d_z = 0.$$

Step 3 – Generate: $T(2.0*e_1 + 0.0*e_2 + 0.0*e_3)$

Step 4 – Build motor:

$$T = 1 - \frac{1}{2}(2e_1)n_\infty = 1 - e_1n_\infty.$$

Step 5 – Sandwich:

$$P' = T P \tilde{T}, \quad \text{down}(P') = (2.0, 0.0, 0.0).$$

Result: RedSphere moves from $[0,0,0]$ to $[2,0,0]$ – its right surface at $x = 3$ touches BlueCube's left surface at $x = 3$. [OK]

Case 2: "Place the green sphere on top of the blue cube."

Step 1 – Parse: "green" → GreenSphere (mover), "blue" → BlueCube (target). Keyword "on top".

Step 2 – Compute:

- BlueCube: center= $[4,0,0]$, max= $[5,1,1]$
- GreenSphere: center= $[-3,0,2]$, size=0.7

$$y_{\text{new}} = y_{\max}^{\text{BlueCube}} + r_{\text{GreenSphere}} = 1.0 + 0.7 = 1.7.$$

$$\Delta = (4 - (-3), 1.7 - 0, 0 - 2) = (7.0, 1.7, -2.0).$$

Step 3 — Generate: $T(7.0 \cdot e1 + 1.7 \cdot e2 + -2.0 \cdot e3)$

Result: GreenSphere moves to $[4, 1.7, 0]$. Bottom at $1.7 - 0.7 = 1.0 = \text{BlueCube top}$. [OK]